APM — 100 QCM avec corrigés / 100 道选择题与解析

Préparation du DST du 22 octobre 2026 / 2026 年 10 月 22 日笔试复习

Questions d'entraînement originales, construites à partir des notions abordées en CM et des TD1, TD2 et TP1. La priorité suit les contenus effectivement étudiés. Les bases des API CUDA et quelques notions OpenMP complètent l'ensemble ; NCCL et l'implémentation complète d'un GEMM optimisé sont hors de cette série. Il ne s'agit pas d'un sujet officiel ni d'une prédiction du sujet.

这是一套依据课程与 TD1、TD2、TP1 整理的自编练习，优先围绕已经学习的内容，补充 CUDA API 基础和少量 OpenMP 概念。本套不涉及 NCCL 或完整的 GEMM 优化实现，也不要求编写完整程序。题目与答案均为复习用参考，不是老师公布的考试题或官方答案。

Chaque question comporte quatre propositions et une seule bonne réponse. La réponse et les explications en français et en chinois figurent directement sous les propositions, avec les repères de cours.

每题四个选项，只有一个正确答案。答案、法语解析和中文解释直接放在本题选项下面，并附有复习讲义链接。

Les numéros tels que 2.4 ou 3.5 désignent les sections du **guide de révision**, et non les numéros des diapositives. Les liens « Repères » mènent à la version chinoise ; la version française conserve les mêmes numéros.

文中"2.4""3.5"等编号指 **复习讲义的小节**，不是课件页码。"复习讲义"链接指向中文版；法语版使用相同编号。

Guide français / 法语复习讲义 · 中文复习讲义 · 100 questions de cours / 100 道课程问答题 · 索引

Parcours prioritaire / 优先复习顺序

Les 60 questions marquées **Prioritaire / 核心** portent sur les notions à consolider d'abord. Les 40 autres complètent la révision ; ce classement n'est pas une probabilité de présence à l'examen.

先做标为 **Prioritaire / 核心** 的 60 道题，剩余 40 道用于补齐细节。这里的"核心"表示复习顺序，不表示一定会考。

Groupe / 分组	Questions prioritaires / 核心题号
1–25 · Exécution, fonctions et indices / 执行模型、函数属性与索引	Q001、Q002、Q003、Q004、Q005、Q006、Q008、Q009、Q010、Q012、Q014、Q015、Q016、Q018、Q023
26–50 · Mémoire et accès aux données / 内存与数据访问	Q026、Q027、Q028、Q029、Q030、Q031、Q032、Q033、Q034、Q035、Q036、Q038、Q040、Q041、Q043
51–75 · Asynchronisme et synchronisation / 异步执行与同步	Q051、Q052、Q053、Q054、Q056、Q058、Q059、Q061、Q063、Q064、Q067、Q068、Q069、Q071、Q072
76–100 · TD, TP et bases des API / TD、TP 与 API 基础	Q076、Q077、Q078、Q079、Q081、Q082、Q083、Q084、Q085、Q087、Q089、Q091、Q092、Q097、Q099

Exécution, fonctions et indices / 执行模型、函数属性与索引

Q001

Prioritaire / 核心

Quel est le principe général d'un accélérateur de calcul ?

计算加速器的一般设计原则是什么?

- **A.** Remplacer systématiquement le processeur qui pilote le programme.

  始终替代负责组织程序执行的主处理器。

- **B.** Supprimer les dépendances présentes dans l'algorithme.

  消除算法中原有的数据依赖。

- **C.** Exécuter toute tâche plus rapidement qu'un CPU, quelle que soit sa taille.

  无论任务大小，都比 CPU 执行得更快。

- **D.** Favoriser une classe de traitements par des choix matériels spécialisés.

  通过专门的硬件取舍，优先支持某一类计算。

Réponse / 答案：D

Un accélérateur consacre ses ressources à certains traitements ; le gain dépend ensuite du travail et de son organisation.

加速器把资源偏向某类处理；实际是否加速，还取决于任务及其组织方式。

Repères / 复习讲义：1.1。

Q002

Prioritaire / 核心

Quel travail expose le plus directement le parallélisme recherché dans les premiers exemples GPU du cours ?

哪类任务最直接体现了课程最初 GPU 示例所需要的并行性?

- **A.** Attendre successivement des réponses interactives de l'utilisateur.

  依次等待用户提供交互式输入。

- **B.** Appliquer la même opération à de nombreux éléments indépendants d'un tableau.

  对数组中大量相互独立的元素执行同一种操作。

- **C.** Calculer un seul élément dont chaque étape dépend strictement de la précédente.

  只计算一个元素，而且每一步都严格依赖上一步。

- **D.** Parcourir une unique chaîne de pointeurs dont chaque adresse dépend de la lecture précédente.

  遍历一条指针链，每个地址都必须通过上一次读取才能获得。

Réponse / 答案：B

Des opérations indépendantes sur beaucoup de données fournissent du travail à de nombreux threads ; cela ne garantit pas à lui seul un gain global.

大量独立的数据操作可以交给许多线程处理；仅有这一条件仍不能保证整个程序加速。

Repères / 复习讲义：1.1。

Q003

Prioritaire / 核心

Dans les exemples CUDA du cours exécutés sur un CPU et un GPU NVIDIA, que désignent « host » et « device » ?

在课程使用 CPU 与 NVIDIA GPU 的 CUDA 示例中，host 和 device 分别指什么?

- **A.** Host : un SM ; device : un warp.

  host：一个 SM；device：一个 warp。
- **B.** Host : un bloc ; device : un thread.

  host：一个线程块；device：一个线程。
- **C.** Host : GPU ; device : CPU.

  host：GPU；device：CPU。
- **D.** Host : CPU ; device : GPU.

  host：CPU；device：GPU。

Réponse / 答案：D

Le host pilote le programme ; le device exécute les calculs GPU qui lui sont confiés.

host 负责组织程序，device 执行交给它的 GPU 计算。

Repères / 复习讲义：1.7。

Q004

Prioritaire / 核心

Dans le programme de TD1, `main` contient `kernel<<<G, B>>>(...)` . Où sont exécutés l'instruction de lancement et le corps du kernel ?

TD1 程序的 `main` 中有 `kernel<<<G, B>>>(...)` 。启动语句和 kernel 函数体分别在哪里执行?

- **A.** L'instruction de lancement sur le CPU, le corps du kernel sur le GPU.

  启动语句在 CPU 上执行，kernel 函数体在 GPU 上执行。
- **B.** L'instruction de lancement sur le GPU, le corps du kernel sur le CPU.

  启动语句在 GPU 上执行，kernel 函数体在 CPU 上执行。
- **C.** Les deux sur le GPU, car la présence de chevrons déplace aussi `main` .

  两者都在 GPU 上执行，因为尖括号也会使 `main` 转移到 GPU。
- **D.** Les deux sur le CPU ; le GPU ne stocke que les données.

  两者都在 CPU 上执行，GPU 只保存数据。

Réponse / 答案：A

Dans ce TD, le code hôte demande le lancement ; chaque thread GPU exécute une instance du kernel.

在这个 TD 中，主机代码发起启动请求；每个 GPU 线程执行 kernel 的一个实例。

Repères / 复习讲义：1.7、1.12。

Q005

Prioritaire / 核心

Quelle description correspond à la hiérarchie logique utilisée pour lancer un kernel dans le modèle de base du cours ?

哪种描述符合课程基础模型中启动 kernel 所使用的逻辑层级?

- **A.** Un thread contient des blocs ; chaque bloc contient des grilles.

  一个线程包含线程块，每个线程块包含网格。

- **B.** Un bloc contient des grilles ; chaque grille contient des threads.

  一个线程块包含网格，每个网格包含线程。

- **C.** Une grille contient des blocs ; chaque bloc contient des threads.

  一个网格包含线程块，每个线程块包含线程。

- **D.** Une grille contient des threads ; chaque thread contient des blocs.

  一个网格包含线程，每个线程包含线程块。

Réponse / 答案：C

Grille et blocs organisent les instances logiques du kernel ; leurs dimensions sont choisies au lancement.

网格和线程块用于组织 kernel 的逻辑执行实例，其维度在启动时指定。

Repères / 复习讲义：1.8。

Q006

Prioritaire / 核心

Dans le modèle de base étudié, quelle relation entre blocs et SM est correcte ?

在课程所学的基础模型中，线程块与 SM 的哪种关系正确?

- **A.** Les threads d'un bloc sont obligatoirement répartis sur tous les SM.

  一个块中的线程必须分散到全部 SM 上。

- **B.** Chaque bloc réserve tout le GPU jusqu'à sa fin.

  每个块都会独占整个 GPU，直到执行结束。

- **C.** Un SM ne peut jamais accueillir plus d'un bloc.

  一个 SM 在任何情况下都不能同时驻留多个块。

- **D.** Un bloc s'exécute sur un même SM ; ce SM peut accueillir plusieurs blocs si ses ressources le permettent.

  一个块在同一个 SM 上执行；资源允许时，一个 SM 可以驻留多个块。

Réponse / 答案：D

Les ressources du SM limitent les blocs résidents ; la grille peut donc être exécutée en plusieurs vagues.

SM 的资源限制了能够驻留的块数，因此网格中的块可以分批执行。

Repères / 复习讲义：1.3。

Q007

Complément / 补充

Quelle affirmation distingue correctement un thread CUDA d'un cœur CUDA ?

哪种说法正确区分了 CUDA 线程与 CUDA 核心?

- **A.** Le thread est une instance logique du kernel ; le cœur est une unité matérielle de calcul.

  线程是 kernel 的逻辑执行实例；核心是硬件计算单元。

- **B.** Le nombre de threads lancés doit être égal au nombre de cœurs matériels.

  启动的线程数必须等于硬件核心数。

- **C.** Chaque thread possède un cœur dédié pendant toute la vie du programme.

  每个线程在程序整个生命周期内都独占一个核心。

- **D.** Les deux termes désignent exactement le même objet physique.

  两个术语指的是完全相同的物理对象。

Réponse / 答案：A

Il n'existe pas de correspondance permanente un thread–un cœur ; les unités sont partagées au fil de l'exécution.

线程与核心之间不存在永久的一一对应；计算单元会在执行过程中供不同线程使用。

Repères / 复习讲义：1.3。

Q008

Prioritaire / 核心

Une grille contient deux blocs de 48 threads chacun. Avec 32 threads par warp, combien de warps sont formés au total ?

一个网格包含两个块，每块 48 个线程。每个 warp 有 32 个线程位置，总共形成多少个 warp?

- **A.** 3 warps.

  3 个 warp。

- **B.** 2 warps.

  2 个 warp。

- **C.** 6 warps.

  6 个 warp。

- **D.** 4 warps.

  4 个 warp。

Réponse / 答案：D

Chaque bloc forme deux warps, dont un partiel. On ne réunit pas des threads de blocs différents pour compléter un warp.

每块形成两个 warp，其中一个不满。不同块的线程不会拼在一起补满一个 warp，因此共 4 个。

Repères / 复习讲义：1.3、1.13。

Q009

Prioritaire / 核心

Dans un même warp, certains threads prennent le `if` et les autres le `else`. Quelle conséquence correspond au modèle SIMT du cours ?

同一个 warp 内，一部分线程进入 `if`，另一部分进入 `else`。哪种后果符合课程介绍的 SIMT 模型?

- **A.** Le programme devient nécessairement incorrect, même sans conflit sur les données.

  即使没有数据冲突，程序也必然不正确。

- **B.** Tous les threads exécutent nécessairement les deux branches et conservent tous leurs résultats.

  所有线程必然执行两个分支，并保留两条路径的所有结果。

- **C.** Le GPU transfère automatiquement ces threads au CPU pour exécuter le branchement.

GPU 会自动把这些线程转交给 CPU 执行分支。

- **D.** Les chemins peuvent progresser séparément avec des ensembles de threads actifs différents.

  不同路径可能分开推进，各路径启用不同的一组线程。

Réponse / 答案：D

La divergence peut coûter du temps, mais un branchement est autorisé et peut être indispensable à la correction.

分歧可能增加时间开销，但条件分支是合法的，也可能是保证正确性所必需的。

Repères / 复习讲义：1.3。

Q010

Prioritaire / 核心

Une grille 1D contient les blocs 0, 1 et 2. Quelle hypothèse est valide sans mécanisme supplémentaire ?

一个一维网格包含编号为 0、1、2 的线程块。在没有额外机制时，哪种判断成立？

- **A.** Le bloc 0 finit obligatoirement avant le début du bloc 1.

  块 0 必须结束后，块 1 才会开始。

- **B.** Le bloc 2 commence obligatoirement avant le bloc 0.

  块 2 必须先于块 0 开始。

- **C.** La numérotation ne garantit pas l'ordre d'exécution des blocs.

  块的编号不保证块的执行顺序。

- **D.** Les trois blocs commencent obligatoirement au même instant.

  三个块必须在完全相同的时刻开始。

Réponse / 答案：C

Les identifiants servent à organiser le travail ; ils ne sont pas une garantie d'ordonnancement entre blocs.

块编号用于组织计算任务，不构成块之间调度顺序的保证。

Repères / 复习讲义：1.13。

Q011

Complément / 补充

Après `dim3 block(16, 8);` , quelles sont les trois composantes de `block` ?

执行 `dim3 block(16, 8);` 后， `block` 的三个分量是什么?

- **A.** (16, 8, 0).

  (16, 8, 0)。
- **B.** (16, 8, 1).

  (16, 8, 1)。
- **C.** (16, 1, 8).

  (16, 1, 8)。
- **D.** (16, 8, 8).

  (16, 8, 8)。

Réponse / 答案：B

Les dimensions non indiquées dans ce constructeur de `dim3` valent 1 ; le bloc contient donc 128 threads.

这个 `dim3` 构造中省略的维度取值为 1，所以该块含 128 个线程。

Repères / 复习讲义：1.8。

Q012

Prioritaire / 核心

Que signifie la déclaration `__host__ __device__` pour une fonction dans le modèle présenté en cours ?

在课程介绍的模型中，为函数声明 `__host__ __device__` 表示什么?

- **A.** Chaque appel exécute nécessairement la fonction simultanément sur le CPU et le GPU.

  每次调用都必然让函数同时在 CPU 与 GPU 上执行。
- **B.** Un appel ordinaire depuis le CPU lance automatiquement une grille GPU.

  CPU 上的普通函数调用会自动启动一个 GPU 网格。
- **C.** Deux versions sont compilées, l’une pour le host et l’autre pour le device.

  编译出两个版本，分别供 host 和 device 执行。
- **D.** La fonction est réservée au GPU malgré la présence de `__host__` .

  即使带有 `__host__` ，该函数也只能在 GPU 上执行。

Réponse / 答案：C

Le lieu de l’appel détermine la version utilisée. Cette double déclaration n’est ni un lancement de kernel ni une exécution simultanée automatique.

调用所在的位置决定使用哪个版本。这种双重声明不等于启动 kernel，也不意味着自动同时在两端执行。

Repères / 复习讲义：2.1。

Q013

Complément / 补充

Un lancement utilise `grid = (3, 2, 1)` et `block = (8, 4, 1)`. Combien de threads sont lancés au total ?

启动配置为 `grid = (3, 2, 1)`、`block = (8, 4, 1)`，总共启动多少个线程?

- **A.** 32.

  32 个。

- **B.** 192.

  192 个。

- **C.** 48.

  48 个。

- **D.** 96.

  96 个。

Réponse / 答案：B

La grille contient 3 × 2 = 6 blocs et chaque bloc 8 × 4 = 32 threads : 6 × 32 = 192.

网格含 3 × 2 = 6 个块，每块含 8 × 4 = 32 个线程，总共 192 个。

Repères / 复习讲义：1.8。

Q014

Prioritaire / 核心

Pour une grille et des blocs 1D, on utilise `i = blockIdx.x * blockDim.x + threadIdx.x`. Avec `blockIdx.x = 3`, `blockDim.x = 64` et `threadIdx.x = 5`, que vaut `i` ?

对于一维网格和一维线程块，使用 `i = blockIdx.x * blockDim.x + threadIdx.x` 。若 `blockIdx.x = 3` 、 `blockDim.x = 64` 、 `threadIdx.x = 5` ， `i` 等于多少?

- **A.** 192.

  192。

- **B.** 320.

  320。

- **C.** 197.

  197。

- **D.** 133.

  133。

Réponse / 答案：C

Trois blocs complets représentent 3 × 64 indices ; on ajoute l'indice local 5, soit 197.

前面三个完整的块贡献 3 × 64 个下标，再加块内编号 5，得到 197。

Repères / 复习讲义：1.9。

Q015

Prioritaire / 核心

On traite `N = 1000` éléments, un élément par thread, avec 128 threads par bloc. Quel est le nombre minimal de blocs pour couvrir tous les éléments ?

处理 `N = 1000` 个元素，每线程一个元素，每块 128 个线程。覆盖全部元素至少需要多少个块?

- **A.** 8.

  8 个。

- **B.** 9.

  9 个。

- **C.** 7.

  7 个。

- **D.** 128.

  128 个。

Réponse / 答案：A

Il faut arrondir 1000/128 au supérieur : 7 blocs couvrent seulement 896 indices, 8 en couvrent 1024.

1000/128 必须向上取整。7 个块只能覆盖 896 个下标，8 个块覆盖 1024 个。

Repères / 复习讲义：1.9。

Q016

Prioritaire / 核心

Pour `N = 1000`, on lance 16 blocs de 64 threads et chaque thread accède à l'élément `i` seulement si `i < N`. Combien de threads n'effectuent pas cet accès ?

对于 `N = 1000`，启动 16 个块，每块 64 个线程；每个线程仅在 `i < N` 时访问元素 `i`。多少个线程不会执行该访问?

- **A.** 23.

  23 个。

- **B.** 0.

  0 个。

- **C.** 24.

  24 个。

- **D.** 64.

  64 个。

Réponse / 答案：C

1024 threads sont lancés ; seuls les indices 0 à 999 sont valides. Les 24 indices restants sont exclus.

总共启动 1024 个线程，合法下标是 0 到 999，剩余 24 个下标被条件排除。

Repères / 复习讲义：1.9。

Q017

Complément / 补充

Un tableau contient 1000 éléments. Avec un élément par thread, `i = blockIdx.x * 256 + threadIdx.x`, 3 blocs de 256 threads et `if (i < 1000)`, que se passe-t-il ?

数组有 1000 个元素，每线程负责一个元素，使用 `i = blockIdx.x * 256 + threadIdx.x`，启动 3 个含 256 线程的块，并检查 `if (i < 1000)`。结果是什么?

- **A.** CUDA crée automatiquement un quatrième bloc.

  CUDA 会自动创建第四个块。

- **B.** Tous les éléments sont traités grâce au test de bornes.

  边界判断会保证全部元素都得到处理。

- **C.** Les éléments 768 à 999 ne sont pas traités.

  元素 768 到 999 没有得到处理。

- **D.** Les éléments 0 à 255 sont traités trois fois et les autres jamais.

  元素 0 到 255 各处理三次，其余元素均不处理。

Réponse / 答案：C

Le lancement fournit seulement les indices 0 à 767 ; un test de bornes n'ajoute pas de threads manquants.

该配置只产生 0 到 767 的下标；边界判断不能补足没有启动的线程。

Repères / 复习讲义：1.9。

Q018

Prioritaire / 核心

Dans les exemples du cours, un kernel `__global__` est lancé depuis le host. Comment fournit-il les résultats du calcul ?

在课程示例中，`__global__` kernel 由 host 启动。它如何提供计算结果?

- **A.** Il retourne nécessairement un `double` contenant la somme des résultats de tous les threads.

  它必然返回一个 `double`，包含所有线程结果的总和。

- **B.** Il doit retourner un tableau C++ directement au point d’appel sur le host.

  它必须直接向 host 调用位置返回一个 C++ 数组。

- **C.** Il retourne `void` et peut écrire les résultats dans les buffers de sortie qui lui sont transmis.

  它返回 `void`，可以把结果写入传入的输出缓冲区。

- **D.** Il ne peut produire aucun résultat exploitable, car son type de retour est `void`.

  因为返回类型是 `void`，所以它不能产生任何可使用的结果。

Réponse / 答案：C

Le type de retour d’un kernel `__global__` est `void`. Les résultats passent par les données écrites ; leur utilisation doit respecter la fin du calcul.

`__global__` kernel 的返回类型是 `void`。结果通过写入的数据传递；使用结果前必须保证相关计算已经完成。

Repères / 复习讲义：2.1。

Q019

Complément / 补充

Une grille a pour dimensions `(5, 3, 1)`. Avec une numérotation à partir de 0 où x varie le plus vite, quel est l'indice linéaire du bloc `(2, 1, 0)` ?

网格维度为 `(5, 3, 1)`，从 0 开始编号，x 变化最快。坐标为 `(2, 1, 0)` 的块，其线性编号是多少?

- **A.** 17.

  17。
- **B.** 11.

  11。
- **C.** 7.

  7。
- **D.** 5.

  5。

Réponse / 答案：C

Une ligne contient 5 blocs : `blockIdx.y * gridDim.x + blockIdx.x = 1 * 5 + 2`.

每行有 5 个块，因此编号为 `blockIdx.y * gridDim.x + blockIdx.x = 1 * 5 + 2`。

Repères / 复习讲义：1.10。

Q020

Complément / 补充

On associe x aux colonnes et y aux lignes. Avec `blockDim = (8, 4, 1)`, `blockIdx = (2, 1, 0)` et `threadIdx = (3, 2, 0)`, quelles coordonnées globales `(x, y)` obtient-on ?

约定 x 对应列、y 对应行。若 `blockDim = (8, 4, 1)`、`blockIdx = (2, 1, 0)`、`threadIdx = (3, 2, 0)`，全局坐标 `(x, y)`

是多少?

- **A.** (11, 6).

  (11, 6)。
- **B.** (5, 3).

  (5, 3)。
- **C.** (6, 19).

  (6, 19)。
- **D.** (19, 6).

  (19, 6)。

Réponse / 答案：D

On applique la formule séparément : x = 2 × 8 + 3 = 19 et y = 1 × 4 + 2 = 6.

分别按各维计算：x = 2 × 8 + 3 = 19，y = 1 × 4 + 2 = 6。

Repères / 复习讲义：1.10。

Q021

Complément / 补充

Pour `gridDim = (3, 2, 2)` et `blockIdx = (1, 1, 1)`, quel est l'indice linéaire du bloc, en partant de 0 et en faisant varier x, puis y, puis z ?

若 `gridDim = (3, 2, 2)`、`blockIdx = (1, 1, 1)`，从 0 开始编号，x 最快、y 次之、z 最慢，该块的线性编号是多少?

- **A.** 10.

  10。
- **B.** 9.

  9。
- **C.** 7.

  7。
- **D.** 11.

  11。

Réponse / 答案：A

L'indice vaut `(blockIdx.z * gridDim.y + blockIdx.y) * gridDim.x + blockIdx.x`, soit (1 × 2 + 1) × 3 + 1 = 10.

编号为 `(blockIdx.z * gridDim.y + blockIdx.y) * gridDim.x + blockIdx.x`，即 (1 × 2 + 1) × 3 + 1 = 10。

Repères / 复习讲义：1.11。

Q022

Complément / 补充

Pour `blockDim = (4, 3, 2)` et `threadIdx = (2, 1, 1)`, quel est l'indice local linéaire, en partant de 0 avec x le plus rapide, puis y, puis z ?

若 `blockDim = (4, 3, 2)`、`threadIdx = (2, 1, 1)`，从 0 开始编号，x 最快、y 次之、z 最慢，块内线性编号是多少?

- **A.** 16.

  16。

- **B.** 18.

  18。

- **C.** 14.

  14。

- **D.** 22.

  22。

Réponse / 答案：B

L'indice vaut `(threadIdx.z * blockDim.y + threadIdx.y) * blockDim.x + threadIdx.x`, soit (1 × 3 + 1) × 4 + 2 = 18.

编号为 `(threadIdx.z * blockDim.y + threadIdx.y) * blockDim.x + threadIdx.x`，即 (1 × 3 + 1) × 4 + 2 = 18。

Repères / 复习讲义：1.11。

Q023

Prioritaire / 核心

On numérote les threads bloc par bloc, à partir de 0, avec x le plus rapide à chaque niveau. Pour `gridDim = (2, 2, 2)`, `blockDim = (4, 2, 2)`, `blockIdx = (1, 0, 1)` et

`threadIdx = (2, 1, 0)`, quel est le numéro global du thread ?

从 0 开始逐块给线程编号，块与块内线程均按 x 最快展开。若 `gridDim = (2, 2, 2)`、`blockDim = (4, 2, 2)`、`blockIdx = (1, 0, 1)`、`threadIdx = (2, 1, 0)`，线程的全局编号是多少?

- **A.** 46.

  46。

- **B.** 86.

  86。

- **C.** 80.

  80。

- **D.** 94.

  94。

Réponse / 答案：B

Le bloc porte l'indice 5 ; il y a 16 threads par bloc et l'indice local vaut 6. Le numéro est 5 × 16 + 6 = 86.

块编号为 5，每块 16 个线程，块内编号为 6，因此全局编号为 5 × 16 + 6 = 86。

Repères / 复习讲义：1.11。

Q024

Complément / 补充

Avec `grid = (3, 2, 1)`, `block = (4, 3, 1)`, `blockIdx = (1, 0, 0)` et `threadIdx = (1, 2, 0)`, le numéro du thread en numérotation par blocs est 21. Si x correspond à la colonne et y à la ligne d'une matrice stockée par lignes de largeur 12, quel indice de tableau faut-il utiliser ?

给定 `grid = (3, 2, 1)`、`block = (4, 3, 1)`、`blockIdx = (1, 0, 0)`、`threadIdx = (1, 2, 0)`，逐块排列得到的线程编号是 21。若 x 对应列、y 对应行，矩阵按行存储且每行 12 个元素，应该使用哪个数组下标?

- **A.** 62.

  62。

- **B.** 21.

21。

- **C.** 17.

  17。

- **D.** 29.

  29。

Réponse / 答案：D

Les coordonnées des données sont x = 5 et y = 2, donc l'indice est 2 × 12 + 5 = 29. La numérotation par blocs suit un autre ordre.

数据坐标为 x = 5、y = 2，因此下标是 2 × 12 + 5 = 29。逐块线程编号使用的是另一种排列顺序。

Repères / 复习讲义：1.10。

Q025

Complément / 补充

Dans un kernel CUDA, un thread peut-il traiter plusieurs éléments d'un tableau ?

在 CUDA kernel 中，一个线程能否处理数组的多个元素?

- **A.** Oui, si le programme organise correctement ses indices et son travail, par exemple avec une boucle.

  可以，只要程序正确组织索引和任务，例如在线程中使用循环。

- **B.** Non, un kernel ne peut contenir aucune boucle.

  不可以，kernel 内不能包含任何循环。

- **C.** Oui, mais uniquement si chaque thread traite tout le tableau en écrivant aux mêmes indices.

  可以，但必须让每个线程都处理整个数组并写入同一批下标。

- **D.** Non, CUDA impose exactement un élément de tableau par thread.

  不可以，CUDA 强制每个线程恰好处理一个数组元素。

Réponse / 答案：A

« Un thread par élément » est une décomposition pédagogique ; le langage permet d'autres répartitions du travail.

“一个线程负责一个元素”是一种教学中的任务划分方式，语言允许其他分工。

Repères / 复习讲义：1.13。

Mémoire et accès aux données / 内存与数据访问

Q026

Prioritaire / 核心

Depuis le code hôte, dans le modèle à mémoires séparées du cours, quelle fonction réalise une allocation explicite dans la mémoire du GPU ?

从主机代码调用，在课程采用的主机与显存分离模型中，哪个函数用于在 GPU 内存中进行显式分配?

- **A.** `cudaMallocHost`
- **B.** `cudaMalloc`
- **C.** `cudaMemcpy`
- **D.** `malloc`

Réponse / 答案：B

`cudaMalloc` alloue un buffer device. `cudaMemcpy` transfère des données entre des buffers déjà alloués.

`cudaMalloc` 分配设备缓冲区； `cudaMemcpy` 在已经分配的缓冲区之间传输数据。

Repères / 复习讲义：2.4。

Q027

Prioritaire / 核心

Que fournit normalement `cudaMallocHost` ?

`cudaMallocHost` 通常分配哪类内存?

- **A.** Une allocation en mémoire partagée propre à un bloc.

  某个线程块独有的共享内存。
- **B.** Une allocation en registres privés du kernel.

  kernel 的线程私有寄存器。
- **C.** Une allocation device équivalente à `cudaMalloc` .

  与 `cudaMalloc` 相同的设备内存。
- **D.** Une allocation hôte dont les pages sont verrouillées.

  主机上的页锁定内存。

Réponse / 答案：D

Il s’agit de mémoire hôte pinned, utile pour les transferts CUDA. Son accessibilité éventuelle par le GPU est une question distincte.

它分配主机 pinned 内存，适合 CUDA 传输。GPU 能否直接访问还要看相应平台和访问条件。

Repères / 复习讲义：2.11。

Q028

Prioritaire / 核心

Sans enregistrement CUDA supplémentaire, comment décrit-on habituellement un buffer hôte obtenu avec `malloc` ?

未额外执行 CUDA 内存注册时，通常怎样描述 `malloc` 得到的主机缓冲区?

- **A.** Une mémoire hôte ordinaire, généralement pageable.

  普通主机内存，通常可分页。

- **B.** Une mémoire hôte toujours pinned.

  一定是页锁定主机内存。

- **C.** Une allocation managed gérée automatiquement par CUDA.

  由 CUDA 自动管理的 managed 分配。

- **D.** Une mémoire constante lisible uniquement par le GPU.

  只有 GPU 能读取的常量内存。

Réponse / 答案：A

`malloc` n’effectue pas lui-même le verrouillage ou l’enregistrement CUDA des pages.

`malloc` 本身不完成 CUDA 所需的锁页或内存注册。

Repères / 复习讲义：2.4、2.11。

Q029

Prioritaire / 核心

Quelle fonction libère correctement un buffer alloué avec `cudaMallocHost` ?

哪一个函数正确释放 `cudaMallocHost` 分配的缓冲区?

- **A.** `free(ptr)`
- **B.** `cudaStreamDestroy(ptr)`

- **C.** `cudaFreeHost(ptr)`
- **D.** `cudaFree(ptr)`

Réponse / 答案：C

Les couples sont `cudaMallocHost` / `cudaFreeHost`， `cudaMalloc` / `cudaFree` et `malloc` / `free`.

分配与释放必须配
对： `cudaMallocHost` / `cudaFreeHost`、 `cudaMalloc` / `cudaFree`、 `malloc` / `free`。

Repères / 复习讲义：2.4、2.11。

Q030

Prioritaire / 核心

On alloue 1 000 valeurs `double`, avec `sizeof(double)=8`. Quelle taille faut-il transmettre à `cudaMalloc` ?

分配 1 000 个 `double`，假设 `sizeof(double)=8`，传给 `cudaMalloc` 的大小是多少?

- **A.** 1 000 octets.

  1 000 字节。
- **B.** 8 000 octets.

  8 000 字节。
- **C.** 8 000 éléments.

  8 000 个元素。
- **D.** 125 octets.

  125 字节。

Réponse / 答案：B

La taille vaut nombre d’éléments multiplié par la taille d’un élément ; le paramètre est exprimé en octets.

字节数等于元素个数乘单个元素的大小；API 的大小参数使用字节。

Repères / 复习讲义：2.4。

Q031

Prioritaire / 核心

Avec `int *p`, `sizeof(int)=4` et `N>1`, de combien d'éléments l'expression `p + N*sizeof(int)` avance-t-elle ? On suppose que le calcul entier ne déborde pas et que le déplacement reste dans les limites du tableau alloué.

设 `int *p`、`sizeof(int)=4` 且 `N>1`，表达式 `p + N*sizeof(int)` 前进了多少个元素？假设整数运算不溢出，且指针偏移仍在已分配数组的合法范围内。

- **A.** 4 éléments indépendamment de N.

  无论 N 是多少，都前进 4 个元素。

- **B.** `4*N` éléments.

  `4*N` 个元素。

- **C.** `N/4` éléments.

  `N/4` 个元素。

- **D.** `N` éléments.

  `N` 个元素。

Réponse / 答案：B

L'arithmétique d'un `int*` compte des int : le déplacement est ici de 4N éléments, soit 16N octets.

`int*` 的指针加法以 int 为单位，因此这里前进 4N 个元素，即 16N 字节。

Repères / 复习讲义：2.4。

Q032

Prioritaire / 核心

Un tableau initialisé sur le CPU doit être copié vers une allocation device. Quelle direction convient ?

将 CPU 上已经初始化的数组复制到设备分配中，应选择哪个方向?

- **A.** `cudaMemcpyDeviceToDevice`
- **B.** `cudaMemcpyHostToHost`
- **C.** `cudaMemcpyDeviceToHost`
- **D.** `cudaMemcpyHostToDevice`

Réponse / 答案：D

La direction décrit la provenance des données puis leur destination : host vers device.

复制方向按数据的来源和去向描述，这里是 host 到 device。

Repères / 复习讲义：2.4。

Q033

Prioritaire / 核心

Pourquoi l’appel C classique `cudaMalloc((void **)&d_a, bytes)` reçoit-il `&d_a` ?

为什么常见的 C 接口调用 `cudaMalloc((void **)&d_a, bytes)` 要传入 `&d_a`？

- **A.** Pour transformer `d_a` en mémoire pinned.

  为了把 `d_a` 转成 pinned 内存。

- **B.** Pour écrire l’adresse de la nouvelle allocation dans la variable `d_a`.

  为了把新分配的地址写回指针变量 `d_a`。

- **C.** Pour indiquer un tableau à deux dimensions.

  为了表示一个二维数组。

- **D.** Pour copier automatiquement le contenu de `d_a` vers le GPU.

  为了自动把 `d_a` 的内容复制到 GPU。

Réponse / 答案：B

Le pointeur est un paramètre de sortie. Une copie ultérieure reçoit les adresses des données, par exemple `d_a` et `h_a`.

这里的指针是输出参数。后续复制接收的是数据地址，例如 `d_a` 和 `h_a`。

Repères / 复习讲义：2.4。

Q034

Prioritaire / 核心

Après un `cudaMalloc` réussi, que peut-on affirmer sur les valeurs du buffer sans autre opération ?

`cudaMalloc` 成功后，未执行其他操作时，可以对缓冲区中的值作何判断?

- **A.** Il faut les initialiser avant de compter sur leur valeur.

  在依赖其值之前需要先初始化。

- **B.** Tous les éléments valent automatiquement zéro.

  所有元素自动为零。

- **C.** Ils contiennent une copie du tableau hôte de même nom.

它们包含同名主机数组的副本。

- **D.** Ils valent tous un tant que le premier kernel n’est pas lancé.

  第一个 kernel 启动前，它们都为一。

Réponse / 答案：A

L’allocation réserve de la mémoire ; elle ne définit pas le contenu initial du tableau.

分配只是保留空间，并不为数组定义初始内容。

Repères / 复习讲义：2.4。

Q035

Prioritaire / 核心

Dans le modèle de blocs ordinaires du cours, qui partage une instance d’un tableau `__shared__` ?

在课程的普通线程块模型中，哪些线程共享同一份 `__shared__` 数组?

- **A.** Les threads d’un même bloc.

  同一个线程块中的线程。

- **B.** Tous les threads de toute la grille.

  整个网格中的所有线程。

- **C.** Le CPU et tous les GPU sans condition.

  CPU 与所有 GPU，无需任何条件。

- **D.** Tous les blocs situés sur un même SM.

  位于同一个 SM 的所有线程块。

Réponse / 答案：A

Chaque bloc possède sa propre instance ; être sur le même SM ne fusionne pas les allocations de deux blocs.

每个块有各自的实例；两个块处于同一 SM，并不会使其共享内存自动合并。

Repères / 复习讲义：2.3。

Q036

Prioritaire / 核心

Quand les données d’une instance ordinaire de mémoire partagée cessent-elles d’être disponibles ?

普通共享内存实例中的数据何时不再有效?

- **A.** Seulement lorsque le processus CPU se termine.

  只有 CPU 进程结束时。

- **B.** Après chaque instruction du kernel.

  kernel 每条指令执行后。

- **C.** À la fin de l'exécution du bloc propriétaire.

  所属线程块执行结束时。

- **D.** Seulement après un appel explicite à `cudaFree`.

  只有显式调用 `cudaFree` 后。

Réponse / 答案：C

La mémoire partagée a la durée de vie du bloc, contrairement à un buffer global explicitement alloué.

共享内存的生命周期与线程块一致，和显式分配的全局缓冲区不同。

Repères / 复习讲义：2.3。

Q037

Complément / 补充

Un premier kernel écrit un buffer global alloué par `cudaMalloc`. Peut-on le réutiliser dans un second kernel ?

第一个 kernel 写入 `cudaMalloc` 分配的全局缓冲区。第二个 kernel 能否继续使用它?

- **A.** Oui, mais il faut d'abord recopier toutes les valeurs sur le CPU.

  可以，但必须先把全部数据复制到 CPU。

- **B.** Seulement si les deux kernels utilisent exactement le même bloc.

  只有两个 kernel 使用完全相同的线程块才可以。

- **C.** Oui, si l'allocation reste valide et si la dépendance entre les kernels est respectée.

  可以，前提是分配仍有效且两个 kernel 的依赖顺序得到保证。

- **D.** Non, toute mémoire globale est détruite à chaque fin de kernel.

  不可以，每次 kernel 结束都会销毁所有全局内存。

Réponse / 答案：C

Le buffer global peut survivre à un lancement. Le conserver sur le GPU évite un aller-retour inutile.

全局缓冲区可以跨越多次 kernel 调用存活，保留在 GPU 上可避免不必要的往返传输。

Repères / 复习讲义：2.3、2.4。

Q038

Prioritaire / 核心

Que signifie « locale » dans l’expression CUDA « mémoire locale » ?

CUDA 中 local memory 的“局部”主要指什么?

- **A.** Elle est située dans la RAM du CPU appelant.

  它位于调用方 CPU 的 RAM。
- **B.** Elle est nécessairement située dans les registres.

  它一定存放在寄存器里。
- **C.** Elle est partagée par tous les threads du bloc.

  它由块内所有线程共享。
- **D.** La mémoire est privée à un thread.

  内存在线程之间是私有的。

Réponse / 答案：D

La portée est privée au thread ; ce nom ne garantit pas une mémoire rapide sur puce.

它的可见范围是单个线程；这个名称不保证它是快速片上存储。

Repères / 复习讲义：2.3。

Q039

Complément / 补充

À ressources SM fixes, quel peut être l’effet d’une forte consommation de registres par thread ?

在 SM 资源固定时，每线程使用大量寄存器可能产生什么影响?

- **A.** Transformer les registres en mémoire partagée entre blocs.

  把寄存器变成块间共享内存。
- **B.** Augmenter automatiquement le nombre de registres physiques du SM.

  自动增加 SM 的物理寄存器总数。
- **C.** Réduire le nombre de blocs pouvant résider simultanément.

减少能够同时驻留的线程块数量。

- **D.** Garantir une occupation de 100 %.

  保证达到 100% 驻留率。

Réponse / 答案：C

Les blocs se partagent un budget de registres fini. La résidence dépend aussi d’autres limites matérielles.

驻留块共同使用有限寄存器资源；能够驻留多少块还受其他硬件限制影响。

Repères / 复习讲义：2.9。

Q040

Prioritaire / 核心

Quel motif est particulièrement favorable à la mémoire constante ?

哪种访问模式尤其适合使用常量内存?

- **A.** Les threads d’un warp lisent 32 adresses constantes différentes au cours d’une même instruction.

  同一 warp 的线程在同一条读取指令中访问 32 个不同的常量地址。

- **B.** Le CPU attend qu’une lecture constante remplace une copie D2H.

  CPU 等待一次常量读取来代替 D2H 复制。

- **C.** Chaque thread écrit simultanément la même variable constante.

  每个线程同时写入同一个常量变量。

- **D.** Les threads d’un warp lisent la même adresse constante.

  同一 warp 的线程读取同一个常量地址。

Réponse / 答案：D

La lecture commune peut bénéficier de la diffusion. Des adresses distinctes peuvent réduire cet avantage.

读取共同地址可以受益于广播；读取不同地址可能削弱这种优势。

Repères / 复习讲义：2.3。

Q041

Prioritaire / 核心

Lequel n’est pas un attribut de déclaration CUDA C++ parmi ces quatre écritures ?

下面哪一项不是 CUDA C++ 的声明属性?

- **A.** `__constant__`
- **B.** `__device__`
- **C.** `__shared__`
- **D.** `__local__`

Réponse / 答案：D

CUDA possède un espace local, mais pas cet attribut `__local__` . Ne pas le confondre avec une écriture OpenCL.

CUDA 有 local memory 存储空间，但没有这里的 `__local__` 声明属性，不能与 OpenCL 的写法混淆。

Repères / 复习讲义：2.1、2.3。

Q042

Complément / 补充

Concernant `__device__` en CUDA C++, quelle proposition est correcte ?

关于 CUDA C++ 的 `__device__` ，哪项正确?

- **A.** Il rend automatiquement la fonction appelable depuis une boucle CPU ordinaire.

  它自动让函数可由普通 CPU 循环调用。
- **B.** Il peut qualifier une fonction device ou une variable device.

  它既可修饰设备函数，也可修饰设备变量。
- **C.** Il impose une allocation en mémoire partagée.

  它强制使用共享内存分配。
- **D.** Il ne peut qualifier que des fonctions, jamais de variables.

  它只能修饰函数，不能修饰变量。

Réponse / 答案：B

La signification dépend de la déclaration : fonction exécutée sur device ou variable résidant sur device.

具体意义取决于声明对象：可以表示设备端执行的函数，也可以表示设备上的变量。

Repères / 复习讲义：2.1、2.3。

Q043

Prioritaire / 核心

Quelle propriété favorise généralement la coalescence des accès à la mémoire globale ?

什么性质通常有利于全局内存访问的合并?

- **A.** Des lanes voisines accèdent à des adresses voisines et correctement alignées.

  相邻 lane 访问邻近且适当对齐的地址。

- **B.** Chaque thread utilise une adresse très éloignée de celle de son voisin.

  每个线程的地址都与相邻线程相距很远。

- **C.** Chaque thread lit deux éléments voisins, indépendamment des autres threads.

  每个线程各自读取两个相邻元素，而不考虑其他线程。

- **D.** On ajoute une synchronisation CPU après chaque lecture.

  每次读取后增加一次 CPU 同步。

Réponse / 答案：A

La coalescence se juge entre les adresses des threads d’un warp pour une même instruction mémoire.

合并访存要看同一条访存指令下，一个 warp 内各线程访问的地址关系。

Repères / 复习讲义：2.7。

Q044

Complément / 补充

Trente-deux threads actifs lisent 32 valeurs contiguës. Peut-on conclure qu’il y aura exactement une transaction mémoire sur tout GPU ?

32 个活跃线程读取 32 个连续值，能否断言在任意 GPU 上都恰好产生一次内存事务?

- **A.** Non ; le type, l’alignement et l’architecture interviennent aussi.

  不能；类型、对齐和架构也会影响事务数。

- **B.** Non ; il y a obligatoirement une transaction distincte par thread.

  不能；每个线程都必然单独产生一次事务。

- **C.** Oui, dès que le tableau est en mémoire globale.

可以，只要数组位于全局内存。

- **D.** Oui ; le nombre 32 impose toujours une transaction unique.

  可以；只要是 32 个线程就固定为一次。

Réponse / 答案：A

La contiguïté est favorable, mais ne fixe pas à elle seule le découpage matériel des transactions.

连续访问是有利条件，但不能单独决定硬件如何划分内存事务。

Repères / 复习讲义：2.7。

Q045

Complément / 补充

Dans un tableau `a` en mémoire globale, pour des lanes voisines d'un warp, que risque de provoquer un grand `stride` dans `a[base + lane*stride]` ?

对于全局内存数组 `a`，同一 warp 的相邻 lane 使用 `a[base + lane*stride]`，较大的 `stride` 可能带来什么影响?

- **A.** Créer automatiquement une copie contiguë en mémoire partagée.

  自动在共享内存中创建连续副本。

- **B.** Synchroniser les threads avant leur lecture.

  在读取前同步这些线程。

- **C.** Disperser les adresses et augmenter le nombre de segments mémoire sollicités.

  使地址分散，增加涉及的内存段数量。

- **D.** Garantir une seule transaction quel que soit l'alignement.

  无论对齐如何，都保证一次事务。

Réponse / 答案：C

Un grand pas entre lanes tend à réduire la coalescence ; le coût exact reste dépendant de la configuration.

lane 之间的较大步长通常不利于合并访存，确切代价还取决于具体配置。

Repères / 复习讲义：2.7。

Q046

Complément / 补充

Une matrice de `float` est stockée par lignes, avec `N>1` colonnes. Les deux éléments considérés sont dans les limites de la matrice. Quel écart sépare les adresses de `a[row*N+col]` et `a[(row+1)*N+col]` ?

一个 `float` 矩阵按行存储，共 N>1 列，以下两个元素都在矩阵范围内。`a[row*N+col]` 与 `a[(row+1)*N+col]` 的地址相隔多少?

- **A.** `N*sizeof(float)` octets.

  `N*sizeof(float)` 字节。

- **B.** N bits.

  N 位。

- **C.** Un nombre d'octets fixé par `gridDim.x` .

  由 `gridDim.x` 固定决定的字节数。

- **D.** `sizeof(float)` octets.

  `sizeof(float)` 字节。

Réponse / 答案：A

Changer de ligne à colonne fixe saute une ligne entière ; changer seulement de colonne saute un élément.

列号不变而行号增加一，需要跨过完整一行；只增加列号才跨过一个元素。

Repères / 复习讲义：2.7。

Q047

Complément / 补充

Dans un modèle simplifié de 32 banques et de mots de 4 octets, la banque vaut `(adresse/4)%32` . Deux mots aux adresses 0 et 128 appartiennent-ils à la même banque ?

采用 32 个 bank、每字 4 字节的简化模型，bank 编号为 `(地址/4)%32` 。地址 0 和 128 的两个字是否位于同一个 bank?

- **A.** On ne peut pas calculer les banques même avec ce modèle.

  即使给出这个模型，也无法计算 bank。

- **B.** Non, ils appartiennent aux banques 0 et 31.

  否，分别位于 bank 0 和 bank 31。

- **C.** Non, ils appartiennent aux banques 0 et 1.

否，分别位于 bank 0 和 bank 1。

- **D.** Oui, ils appartiennent tous deux à la banque 0.

  是，都位于 bank 0。

Réponse / 答案：D

`0/4%32=0` et `128/4%32=0`. Ce sont des mots distincts ; des accès simultanés peuvent donc entrer en conflit.

`0/4%32=0`， `128/4%32=0`。它们是同一 bank 的不同字，同时访问可能发生冲突。

Repères / 复习讲义：2.8。

Q048

Complément / 补充

Pourquoi plusieurs threads d'un même warp lisant exactement le même mot partagé au cours d'une même instruction ne provoquent-ils pas nécessairement un conflit de banques ?

为什么同一 warp 的多个线程在同一条读取指令中读取共享内存中完全相同的一个字，不一定产生 bank conflict?

- **A.** La mémoire partagée ne possède aucune banque.

  共享内存没有 bank。

- **B.** Le matériel peut diffuser une même valeur aux lecteurs.

  硬件可能把同一个值广播给这些线程。

- **C.** La lecture transforme automatiquement le mot en registre partagé entre blocs.

  读取会自动把这个字变成块间共享寄存器。

- **D.** Les threads se synchronisent implicitement avec tous les blocs.

  这些线程会隐式同步整个网格。

Réponse / 答案：B

La diffusion d'un même mot se distingue de demandes visant plusieurs mots différents d'une même banque.

广播同一个字，与读取同一 bank 中多个不同字是不同情况。

Repères / 复习讲义：2.8。

Q049

Complément / 补充

Que représente le `pitch` retourné par `cudaMallocPitch` ?

`cudaMallocPitch` 返回的 `pitch` 表示什么?

- **A.** La taille totale de la matrice en octets.

  整个矩阵的总字节数。

- **B.** Le nombre de colonnes utiles de la matrice.

  矩阵有效列数。

- **C.** La distance en octets entre les débuts de deux lignes successives.

  相邻两行起始地址之间的字节距离。

- **D.** Uniquement le nombre d’octets de padding à la fin d’une ligne.

  仅表示一行末尾填充的字节数。

Réponse / 答案：C

Le pitch inclut la largeur utile et l’éventuel padding. La largeur utile et la distance entre lignes peuvent différer.

pitch 包含有效行宽和可能存在的填充，因此实际行间距可以大于有效行宽。

Repères / 复习讲义：2.5。

Q050

Complément / 补充

Une allocation 2D retourne `pitch=512` octets. Quel est le décalage du début de la ligne d’indice 3, les lignes étant numérotées à partir de 0 ?

二维分配返回 `pitch=512` 字节，行号从 0 开始。第 3 行起点相对数组基址的偏移是多少?

- **A.** 1 536 octets.

  1 536 字节。

- **B.** 1 536 éléments, quel que soit leur type.

  1 536 个元素，与元素类型无关。

- **C.** 3 octets.

  3 字节。

- **D.** 512 octets.

  512 字节。

Réponse / 答案：A

Le décalage en octets vaut `row*pitch`, donc `3*512=1536`.

字节偏移为 `row*pitch`，因此是 `3*512=1536`。

Repères / 复习讲义：2.5。

Asynchronisme et synchronisation / 异步执行与同步

Q051

Prioritaire / 核心

Que peut-on conclure lorsque le CPU revient d'un lancement CUDA `K<<<G,B>>>(...)`, sans synchronisation ultérieure ?

CPU 执行完 CUDA 启动语句 `K<<<G,B>>>(...)` 后，尚未执行后续同步，此时能得出什么结论?

- **A.** Le noyau n'est pas nécessairement terminé.

  kernel 不一定已经完成。

- **B.** Tous les autres streams sont terminés.

  其他所有 stream 都已完成。

- **C.** Le résultat est automatiquement copié sur le CPU.

  结果已经自动复制到 CPU。

- **D.** Tous les threads du noyau ont terminé.

  kernel 中的所有线程都已完成。

Réponse / 答案：A

Le lancement est asynchrone par rapport au CPU. Le retour de l'appel ne garantit pas la fin du calcul GPU.

kernel 启动相对于 CPU 是异步的；启动语句返回不能保证 GPU 计算已经结束。

Repères / 复习讲义：2.11。

Q052

Prioritaire / 核心

`A` écrit `d_x` et `B` lit ensuite `d_x`. Les lancements réussissent et utilisent le même stream explicite `s`. Quel ordre est garanti ?

`A` 写入 `d_x`，`B` 随后读取 `d_x`。下面的启动均成功，并使用同一显式 stream `s`。哪一种顺序有保证?

```
A<<<G, Bsize, 0, s>>>(d_x);
B<<<G, Bsize, 0, s>>>(d_x);
```

- **A.** B utilise les résultats de A après la fin de A, grâce à l'ordre du stream.

  依靠同一 stream 的顺序，B 在 A 完成后使用其结果。
- **B.** B peut lire d_x avant la fin de A car les lancements sont asynchrones.

  因为启动是异步的，B 可以在 A 完成前读取 d_x。
- **C.** CUDA choisit cet ordre en analysant les noms des pointeurs.

  CUDA 通过分析指针名称决定这一顺序。
- **D.** Le CPU doit synchroniser après A pour établir cet ordre.

  必须由 CPU 在 A 后同步才能建立这一顺序。

Réponse / 答案：A

Les opérations d'un même stream sont ordonnées. L'asynchronisme vis-à-vis du CPU ne supprime pas cet ordre GPU.

同一 stream 中的操作有先后顺序。相对于 CPU 异步，不等于同一 stream 内也可以任意乱序。

Repères / 复习讲义：2.11。

Q053

Prioritaire / 核心

Les buffers sont valides, `h_out` est pinned et tous les appels réussissent. Après ces instructions, que peut faire le CPU ?

缓冲区有效，`h_out` 是 pinned 主机内存，所有调用均成功。执行下面的指令后，CPU 可以做什么?

```
K<<<G, B, 0, s>>>(d_out);
cudaMemcpyAsync(h_out, d_out, nbytes, cudaMemcpyDeviceToHost, s);
cudaStreamSynchronize(s);
```

- **A.** Lire h_out seulement après un second cudaDeviceSynchronize.

  还必须再执行一次 cudaDeviceSynchronize 才能读取 h_out。

- **B.** Supposer que K n'a pas encore commencé.

  认定 K 仍未开始执行。

- **C.** Lire d_out directement comme un tableau CPU ordinaire.

  把 d_out 直接作为普通 CPU 数组读取。

- **D.** Lire h_out : le noyau puis la copie ont terminé.

  读取 h_out，因为 kernel 和后续复制都已结束。

Réponse / 答案：D

Le noyau et la copie sont dans le même stream. L'attente placée après la copie garantit la disponibilité de la destination côté CPU.

kernel 与复制位于同一 stream，复制后的等待保证回传完成，因此 CPU 可以读取 h_out。

Repères / 复习讲义：2.11。

Q054

Prioritaire / 核心

Un noyau producteur est soumis dans `s1`, puis un noyau consommateur dans `s2`. Les streams sont distincts et aucune dépendance implicite ou explicite ne les relie. Le consommateur lit les données écrites par le producteur. Quelle affirmation est correcte ?

生产者 kernel 提交到 `s1`，随后消费者 kernel 提交到 `s2`。两个 stream 不同，彼此没有隐式或显式依赖；消费者读取生产者写入的数据。哪个说法正确?

- **A.** Un accès à la mémoire globale attend automatiquement son producteur.

  访问全局内存时会自动等待对应生产者完成。

- **B.** Il faut établir une dépendance avant que le consommateur lise ces données.

  需要在消费者读取这些数据之前建立依赖。

- **C.** L'ordre des deux appels CPU suffit à ordonner les deux kernels.

  两个 CPU 启动调用的先后顺序足以保证两个 kernel 的先后顺序。

- **D.** CUDA détecte automatiquement la dépendance à partir du pointeur partagé.

  CUDA 会根据共用的指针自动检测出依赖。

Réponse / 答案：B

Les streams distincts ne découvrent pas automatiquement les dépendances de données. Une attente d'événement ou une synchronisation correctement placée peut établir l'ordre requis.

不同 stream 不会自动发现数据依赖；需要正确放置 event 等待或同步来建立先后关系。

Repères / 复习讲义：2.11。

Q055

Complément / 补充

Deux noyaux indépendants sont placés dans deux streams distincts. Que peut-on affirmer sur leur exécution ?

两个相互独立的 kernel 被放在两个不同的 stream 中。对于它们的执行，可以确认什么?

- **A.** Ils peuvent se chevaucher si le matériel et les ressources le permettent.

  硬件和资源允许时，它们可以重叠执行。

- **B.** Ils utilisent obligatoirement deux GPU différents.

  它们必定使用两个不同的 GPU。

- **C.** Ils s'exécutent obligatoirement au même instant.

  它们必定同时执行。

- **D.** Le second stream est interdit tant que le premier n'est pas vide.

  只要第一个 stream 未清空，就禁止使用第二个 stream。

Réponse / 答案：A

Plusieurs streams rendent un recouvrement possible, mais les capacités matérielles, les ressources et l'ordonnancement déterminent s'il a lieu.

多个 stream 提供重叠执行的可能性，实际能否重叠取决于硬件能力、资源和调度。

Repères / 复习讲义：2.11。

Q056

Prioritaire / 核心

Depuis le CPU, que garantit le retour réussi de `cudaDeviceSynchronize()` pour le device et le contexte courants ?

CPU 调用 `cudaDeviceSynchronize()` 并成功返回时，对当前 device 和当前 context 有什么保证?

- **A.** Tous les GPU de tous les processus ont terminé.

所有进程使用的所有 GPU 都已经完成。

- **B.** Les travaux précédemment soumis dans ce contexte sur ce device sont terminés.

  此前提交到该 device 当前 context 中的工作已经完成。

- **C.** Seul le dernier thread lancé a terminé.

  只有最后启动的一个线程已经完成。

- **D.** Toutes les données du device ont été copiées vers le CPU.

  device 上的全部数据都已复制到 CPU。

Réponse / 答案：B

Cette synchronisation attend les travaux antérieurs du contexte courant sur le device courant. Elle n'effectue pas à elle seule une copie de résultats.

它等待当前 device、当前 context 中此前的工作；同步本身不等于把结果复制回 CPU。

Repères / 复习讲义：2.11。

Q057

Complément / 补充

`s1` et `s2` sont indépendants, sans dépendance entre eux. Les appels réussissent. Que garantit la dernière ligne ?

`s1` 与 `s2` 相互独立，没有依赖，调用均成功。下面代码的最后一行保证什么？

```
A<<<G, B, 0, s1>>>(d_a);
Bkernel<<<G, B, 0, s2>>>(d_b);
cudaStreamSynchronize(s1);
```

- **A.** Bkernel est terminé ; la fin de A n'est pas garantie.

  Bkernel 已完成；不保证 A 已完成。

- **B.** A est terminé ; la fin de Bkernel n'est pas garantie par cette attente.

  A 已完成；这次等待不保证 Bkernel 已完成。

- **C.** Les deux noyaux sont nécessairement terminés.

  两个 kernel 都必定已经完成。

- **D.** A ne peut commencer qu'après le retour de cette attente.

  A 只能在这次等待返回后开始执行。

Réponse / 答案：B

`cudaStreamSynchronize(s1)` attend le stream indiqué. Sans dépendance, cette attente ne constitue pas une attente de s2.

cudaStreamSynchronize(s1) 等待指定的 stream；在不存在依赖时，这不等于等待 s2。

Repères / 复习讲义：2.11。

Q058

Prioritaire / 核心

Un événement valide `e` est enregistré après un noyau dans le stream `s` avec `cudaEventRecord(e, s)`. Que représente cette opération ?

在 stream `s` 中某个 kernel 后调用 `cudaEventRecord(e, s)`，其中 `e` 是有效事件。这个操作表示什么?

- **A.** Elle place un repère dans s dont l'achèvement suit les opérations précédentes de s.

  它在 s 中放置一个标记，该标记在 s 中此前的操作完成后才完成。

- **B.** Elle attend obligatoirement la fin du noyau sur le CPU avant de retourner.

  它必定在 CPU 上等待 kernel 结束后才返回。

- **C.** Elle crée automatiquement un nouveau stream pour le noyau.

  它自动为 kernel 创建一个新的 stream。

- **D.** Elle copie les résultats du noyau dans l'événement.

  它将 kernel 的计算结果复制进事件。

Réponse / 答案：A

Enregistrer un événement et attendre cet événement sont deux opérations différentes. Le record sert à marquer une progression dans le stream.

记录事件与等待事件是两个不同操作；record 用来标记 stream 的执行进度。

Repères / 复习讲义：2.11、2.12。

Q059

Prioritaire / 核心

`e` a déjà été enregistré après le producteur. Quelle API permet de faire attendre les opérations suivantes du stream consommateur `s` sans demander au CPU d'attendre la fin de `e` ?

事件 `e` 已在生产者之后记录。哪个 API 可以让消费者 stream `s` 的后续操作等待 `e`，而不要求 CPU 等待 `e` 完成?

- **A.** `cudaStreamWaitEvent(s, e, 0)`
- **B.** `cudaDeviceSynchronize()`
- **C.** `cudaStreamSynchronize(s)`
- **D.** `cudaEventSynchronize(e)`

Réponse / 答案：A

`cudaStreamWaitEvent` établit une dépendance dans le stream. Les trois autres appels sont des attentes effectuées par le thread CPU appelant.

cudaStreamWaitEvent 在 stream 中建立依赖；其他三个调用会让发起调用的 CPU 线程等待相应工作完成。

Repères / 复习讲义：2.11、2.12。

Q060

Complément / 补充

Après l'achèvement de deux événements de mesure valides, dans quelle unité `cudaEventElapsedTime(&t, start, stop)` écrit-il la durée `t` ?

两个有效计时事件都已完成时，`cudaEventElapsedTime(&t, start, stop)` 写入的时间 `t` 使用什么单位?

- **A.** Millisecondes.

  毫秒。
- **B.** Cycles d'horloge GPU.

  GPU 时钟周期。
- **C.** Secondes.

  秒。
- **D.** Nanosecondes.

  纳秒。

Réponse / 答案：A

L'argument de sortie de `cudaEventElapsedTime` est une durée en millisecondes.

cudaEventElapsedTime 的输出时间单位是毫秒。

Repères / 复习讲义：2.12。

Q061

Prioritaire / 核心

Les événements sont valides pour la mesure du temps et les trois opérations utilisent le même stream. Avant de demander une durée complète, quelle attente CPU convient ?

两个事件支持计时，三项操作都在同一个 stream 中。为了随后读取完整区间的耗时，应该采用哪个 CPU 等待操作?

```
cudaEventRecord(start, s);
K<<<G, B, 0, s>>>(d_x);
cudaEventRecord(stop, s);
```

- **A.** Aucune attente : le retour de cudaEventRecord(stop, s) prouve que K est terminé.

  无需等待：cudaEventRecord(stop, s) 返回就证明 K 已完成。
- **B.** `__syncthreads()` appelé depuis le CPU.

  在 CPU 端调用 __syncthreads()。
- **C.** `cudaEventSynchronize(stop)`
- **D.** Attendre uniquement start, car cela garantit la fin de K.

  只等待 start，因为这就能保证 K 完成。

Réponse / 答案：C

Le stop suit le noyau dans s. Attendre son achèvement garantit que le noyau et l’intervalle mesuré sont terminés ; attendre start ne suffit pas.

stop 位于同流 kernel 之后，等待它完成可保证 kernel 和被测区间均已结束；仅等待 start 不够。

Repères / 复习讲义：2.12。

Q062

Complément / 补充

Un chronomètre CPU est démarré juste avant un lancement de noyau et arrêté immédiatement après, sans aucune attente GPU. Pourquoi cette durée ne mesure-t-elle pas nécessairement le temps complet du noyau ?

CPU 计时器在启动 kernel 前开始、启动语句返回后立即停止，期间没有任何 GPU 等待。为什么这个时间不一定是 kernel 的完整执行时间?

- **A.** Les noyaux ne peuvent jamais être mesurés.

  kernel 的时间无法测量。

- **B.** Le lancement convertit automatiquement les secondes en cycles.

  启动语句会自动把秒换算为时钟周期。

- **C.** Un chronomètre CPU retourne toujours zéro pour un programme CUDA.

  CPU 计时器对 CUDA 程序总是返回零。

- **D.** Le CPU peut arrêter le chronomètre avant la fin du noyau.

  CPU 可能在 kernel 完成之前就停止计时。

Réponse / 答案：D

Un lancement est asynchrone. Pour inclure la fin du travail dans une mesure CPU, il faut attendre cette fin avant de prendre le second horodatage.

kernel 启动是异步的。使用 CPU 计时器测完整计算时，结束计时前要等待计算完成。

Repères / 复习讲义：2.12。

Q063

Prioritaire / 核心

`h` est un buffer pinned valide et la copie H→D ci-dessous est encore en cours lorsque le CPU atteint la deuxième ligne. Quelle est l'erreur ?

`h` 是有效的 pinned 缓冲区；CPU 到达第二行时，下面的 H→D 复制仍在进行。问题是什么?

```
cudaMemcpyAsync(d, h, nbytes, cudaMemcpyHostToDevice, s);
h[0] = 42;
```

- **A.** Le CPU modifie une source que la copie peut encore lire.

  CPU 修改了复制操作仍可能读取的源数据。

- **B.** Toute copie H→D modifie automatiquement la source.

  每次 H→D 复制都会自动修改源数据。

- **C.** Il faut remplacer h[0] par d[0] dans le code CPU.

  应该把 CPU 代码中的 h[0] 改为 d[0]。

- **D.** Une mémoire pinned ne peut jamais être écrite par le CPU.

  pinned 内存永远不能由 CPU 写入。

Réponse / 答案：A

La source d’un transfert asynchrone doit rester valide et inchangée jusqu’à la fin de sa lecture par le transfert.

异步传输完成对源缓冲区的读取前，源数据必须保持有效且不被改动。

Repères / 复习讲义：2.11。

Q064

Prioritaire / 核心

`h` est pinned et le transfert ci-dessous n’est pas encore terminé au moment de l’affichage. Quel diagnostic est correct ?

`h` 是 pinned 内存，执行下面的打印语句时 D→H 复制还未完成。哪种判断正确?

```
cudaMemcpyAsync(h, d, nbytes, cudaMemcpyDeviceToHost, s);
printf("%d\n", h[0]);
```

- **A.** L’affichage attend automatiquement le stream s.

  printf 会自动等待 stream s。
- **B.** Une mémoire pinned contient toujours la nouvelle valeur dès la soumission.

  pinned 内存在提交复制时就一定已经包含新值。
- **C.** La lecture de h[0] est prématurée ; il faut attendre la fin du transfert.

  读取 h[0] 过早，需要等待复制完成。
- **D.** La copie D→H est impossible avec un buffer pinned.

  使用 pinned 缓冲区无法执行 D→H 复制。

Réponse / 答案：C

Le caractère pinned permet le transfert asynchrone mais ne garantit pas sa fin. Le CPU doit attendre avant de lire la destination.

pinned 属性支持异步传输，却不保证传输已经完成；CPU 读取目标缓冲区前需要等待。

Repères / 复习讲义：2.11。

Q065

Complément / 补充

Une copie asynchrone utilise encore un buffer hôte pinned. À quel moment peut-on libérer ou réutiliser ce buffer en sécurité, en supposant qu'aucune autre opération ne l'utilise ?

异步复制仍在使用一个 pinned 主机缓冲区。假设没有其他操作使用它，何时可以安全释放或复用这个缓冲区?

- **A.** Dès que l'adresse du buffer a été imprimée.

  一打印出缓冲区地址就可以。

- **B.** Dès qu'un autre stream a terminé un noyau indépendant.

  另一个 stream 中独立的 kernel 一完成就可以。

- **C.** Après avoir établi que la copie est terminée.

  确认复制完成之后。

- **D.** Dès que l'appel de soumission retourne, dans tous les cas.

  在任何情况下，提交调用一返回就可以。

Réponse / 答案：C

La durée de vie du buffer doit couvrir tout le transfert. Une attente du stream concerné, par exemple, peut en établir l'achèvement.

缓冲区的生命周期必须覆盖整个传输过程；例如等待相关 stream 完成，就可以确认传输已结束。

Repères / 复习讲义：2.11。

Q066

Complément / 补充

Quelle affirmation est correcte concernant le suffixe `Async` des API de copie CUDA ?

关于 CUDA 复制 API 的 `Async` 后缀，哪个说法正确?

- **A.** Il supprime toute nécessité de gérer les dépendances de données.

  它消除了管理数据依赖的必要性。

- **B.** Il garantit toujours un retour immédiat, quel que soit le buffer hôte.

  无论主机缓冲区类型如何，都保证立即返回。

- **C.** Il ne suffit pas à garantir un retour immédiatement non bloquant dans toutes les situations ; le type de mémoire intervient.

  不能只凭该后缀保证所有情况下都立即非阻塞返回，内存类型会影响行为。

- **D.** Il garantit que la copie est terminée dès le retour.

它保证调用返回时复制已经结束。

Réponse / 答案：C

La mémoire hôte pageable peut entraîner du staging ou des blocages. Le mot Async ne remplace ni les conditions du transfert ni les synchronisations nécessaires.

普通可分页主机内存可能引入中间复制或阻塞；Async 名称不能替代对传输条件和同步要求的分析。

Repères / 复习讲义：2.11。

Q067

Prioritaire / 核心

Les streams sont distincts, les objets sont valides et les appels réussissent. `B` consomme ce que `A` produit dans `d_x` . Que garantit ce code ?

两个 stream 不同，对象有效，调用均成功。`B` 使用 `A` 写入 `d_x` 的结果。下面的代码保证什么？

```
A<<<G, T, 0, s1>>>(d_x);
cudaEventRecord(e, s1);
cudaStreamWaitEvent(s2, e, 0);
B<<<G, T, 0, s2>>>(d_x);
```

- **A.** A attend la fin de B.

  A 等待 B 完成。

- **B.** Le CPU attend nécessairement A avant de soumettre B.

  CPU 必定等待 A 完成后才提交 B。

- **C.** A et B deviennent un seul noyau.

  A 和 B 合并为一个 kernel。

- **D.** B attend la fin de A grâce à l’événement enregistré après A.

  借助 A 之后记录的事件，B 等待 A 完成。

Réponse / 答案：D

L’événement est enregistré après le producteur et son attente précède le consommateur. Ce placement établit A → B dans les files GPU.

事件记录放在生产者之后，事件等待放在消费者之前，因此在 GPU 队列中建立了 A → B 的依赖。

Repères / 复习讲义：2.11。

Q068

Prioritaire / 核心

`P` écrit `d_x`, `C` le lit ; `s1` et `s2` sont distincts et sans dépendance implicite. Le code suivant assure-t-il que C lit les résultats de P ?

`P` 写入 `d_x`，`C` 读取 `d_x`；`s1`、`s2` 不同，且没有隐式依赖。下面的代码能否保证 C 读取 P 产生的结果?

```
P<<<G, T, 0, s1>>>(d_x);
C<<<G, T, 0, s2>>>(d_x);
cudaDeviceSynchronize();
```

- **A.** Oui : cudaDeviceSynchronize réexécute C si nécessaire.

  能：cudaDeviceSynchronize 会在必要时重新执行 C。
- **B.** Oui : l'ordre des appels CPU suffit entre tous les streams.

  能：CPU 调用顺序对所有 stream 都足够。
- **C.** Non : cudaDeviceSynchronize ne peut attendre aucun noyau.

  不能：cudaDeviceSynchronize 无法等待任何 kernel。
- **D.** Non : attendre les deux à la fin n'impose pas rétroactivement P avant C.

  不能：末尾等待两者完成，不会追溯建立 P 先于 C 的执行顺序。

Réponse / 答案：D

La synchronisation finale attend l'achèvement ; elle ne répare pas une lecture prématurée déjà effectuée. La dépendance doit être établie avant le consommateur.

末尾同步只等待完成，无法修复此前已经发生的过早读取；依赖必须在消费者执行前建立。

Repères / 复习讲义：2.11。

Q069

Prioritaire / 核心

Dans un noyau ordinaire, quelle est la portée de `__syncthreads()` ?

在普通 kernel 中，`__syncthreads()` 的同步范围是什么?

- **A.** Les threads d'un même bloc.

  同一个 block 中的线程。

- **B.** Tous les streams du GPU.

  GPU 上的所有 stream。

- **C.** Le CPU et tous les GPU de la machine.

  CPU 和机器上的所有 GPU。

- **D.** Tous les blocs de la grille.

  整个 grid 中的所有 block。

Réponse / 答案：A

`__syncthreads()` est une barrière de bloc. Elle ne constitue pas une barrière globale entre tous les blocs.

__syncthreads() 是 block 内的屏障，不是所有 block 之间的全局屏障。

Repères / 复习讲义：2.10。

Q070

Complément / 补充

Un bloc unidimensionnel comporte exactement `T > 0` threads (`blockDim = (T, 1, 1)`) et `tile` possède `T` cases shared. Tous les threads exécutent ces lignes. Que garantit la barrière ?

一个一维 block 恰有 `T > 0` 个线程（`blockDim = (T, 1, 1)`），`tile` 是含 `T` 个元素的 shared 数组。所有线程都执行下面的语句。屏障保证什么?

```
tile[threadIdx.x] = input[threadIdx.x];
__syncthreads();
int v = tile[(threadIdx.x + 1) % T];
```

- **A.** Les écritures précédentes de tile par le bloc sont terminées et visibles avant ces lectures.

  block 内先前对 tile 的写入在后续读取前完成，并对这些线程可见。

- **B.** Tous les blocs du GPU ont fini leur chargement.

  GPU 上所有 block 都已完成加载。

- **C.** Chaque lecture retourne obligatoirement la valeur du même thread.

  每次读取都必定返回当前线程自己写入的值。

- **D.** Les threads sont tous exécutés par le CPU après la barrière.

  屏障之后所有线程都改由 CPU 执行。

Réponse / 答案：A

La barrière établit ici le rendez-vous du bloc et la visibilité des écritures partagées avant la lecture des cases voisines.

这里屏障使 block 内线程会合，并保证共享内存写入对之后读取相邻元素的线程可见。

Repères / 复习讲义：2.10。

Q071

Prioritaire / 核心

Un bloc de dimensions `blockDim = (128, 1, 1)` contient 128 threads actifs ; aucun ne quitte le noyau avant ce code. Pourquoi cette utilisation est-elle incorrecte ?

一个尺寸为 `blockDim = (128, 1, 1)` 的 block 中有 128 个活动线程，且此前没有线程退出 kernel。为什么这种写法不正确?

```
if (threadIdx.x < 64) {
    __syncthreads();
}
```

- **A.** La barrière doit toujours être exécutée uniquement par le thread 0.

  屏障永远只能由线程 0 执行。

- **B.** Toute instruction if est interdite dans un noyau.

  kernel 中禁止出现任何 if 语句。

- **C.** La condition n’est pas uniforme : une partie du bloc atteint la barrière et l’autre la saute.

  条件在 block 内不一致，部分线程到达屏障，另一部分跳过屏障。

- **D.** Un bloc CUDA ne peut jamais contenir 128 threads.

  CUDA block 永远不允许包含 128 个线程。

Réponse / 答案：C

Une barrière de bloc ne doit pas être placée dans une branche prise seulement par une partie des threads du bloc.

不能把 block 级屏障放在仅部分线程进入的分支中；各线程需要一致地参与相应屏障。

Repères / 复习讲义：2.10。

Q072

Prioritaire / 核心

Que garantit un `atomicAdd` sur une adresse entière supportée, lorsque tous les threads participants sont sur le même GPU et que tous les accès concurrents à cette adresse utilisent la même opération atomique ?

对于支持的整数地址，所有参与线程都在同一 GPU 上，且对该地址的所有并发访问都使用相同的原子加操作时，`atomicAdd` 保证什么?

- **A.** Toutes les opérations mémoire du programme sont automatiquement ordonnées.

  程序中所有内存操作都会自动建立完整顺序。

- **B.** Toutes les copies asynchrones vers le CPU sont terminées.

  所有回传到 CPU 的异步复制都已完成。

- **C.** Tous les threads de la grille attendent à cet endroit.

  grid 中所有线程都会在这里会合等待。

- **D.** L'ajout sur cette adresse est indivisible vis-à-vis de ces autres ajouts atomiques.

  对该地址的加法更新相对于其他这些原子加操作不可分割。

Réponse / 答案：D

L'atomicité protège la mise à jour de la cible. Un atomicAdd n'est ni une barrière collective ni une synchronisation générale de toutes les opérations.

原子性保护目标地址上的这次更新；atomicAdd 既不是集体屏障，也不是对全部操作的一般同步。

Repères / 复习讲义：2.10。

Q073

Complément / 补充

Un entier device `counter` vaut 0. Cent threads exécutent chacun `counter++` sans opération atomique ni autre protection. Après la fin du noyau, quelle conclusion est correcte ?

device 上的整数 `counter` 初始为 0。100 个线程各执行一次 `counter++`，没有使用原子操作或其他保护。kernel 完成后，哪个结论正确?

- **A.** Le résultat vaut nécessairement 100.

  结果必定等于 100。

- **B.** La valeur 100 n'est pas garantie, car des mises à jour concurrentes peuvent être perdues.

  不保证结果为 100，因为并发更新可能丢失。

- **C.** La fin du noyau rend rétroactivement les incréments atomiques.

kernel 结束会追溯地使这些递增操作变为原子操作。

- **D.** Le résultat vaut nécessairement 1.

  结果必定等于 1。

Réponse / 答案：B

Une incrémentation ordinaire combine lecture, addition et écriture. Plusieurs threads peuvent lire la même ancienne valeur puis écraser leurs mises à jour.

普通递增包含读取、加法和写回；多个线程可能读取相同旧值，再彼此覆盖更新。

Repères / 复习讲义：2.10。

Q074

Complément / 补充

Un entier device `counter` est initialisé à 0. Exactement 100 threads exécutent chacun une fois `atomicAdd(&counter, 1)` ; aucun autre accès ne le modifie, et il n'y a pas de débordement. Quelle valeur obtient-on après la fin réussie du noyau ?

device 整数 `counter` 初始为 0，恰好 100 个线程各执行一次 `atomicAdd(&counter, 1)`，没有其他修改，也不发生整数溢出。kernel 成功完成后，结果是什么?

- **A.** Une valeur arbitraire entre 0 et 100 car les ajouts sont concurrents.

  由于并发相加，结果是 0 到 100 之间的任意值。

- **B.** 1, quel que soit le nombre de threads.

  无论有多少线程都为 1。

- **C.** 100.

  100。

- **D.** 0, car atomicAdd ne modifie que les registres locaux.

  0，因为 atomicAdd 只修改线程局部寄存器。

Réponse / 答案：C

L'atomicité empêche la perte d'incréments sur ce compteur entier. Les 100 mises à jour sont toutes prises en compte.

原子加避免这个整数计数器上的递增丢失，100 次更新都会计入结果。

Repères / 复习讲义：2.10。

Q075

Complément / 补充

Plusieurs threads effectuent des lectures et écritures concurrentes non protégées sur une variable. Ajouter seulement `volatile` suffit-il à corriger ce problème de synchronisation ?

多个线程对一个变量进行未受保护的并发读写。仅添加 `volatile` 是否足以解决这个同步问题?

- **A.** Oui : volatile équivaut à cudaDeviceSynchronize.

  足够：volatile 等价于 cudaDeviceSynchronize。

- **B.** Non : volatile ne fournit ni atomicité ni barrière entre threads.

  不够：volatile 不提供原子性，也不提供线程间屏障。

- **C.** Oui : volatile force automatiquement tous les threads à s'exécuter l'un après l'autre.

  足够：volatile 自动强制所有线程逐个执行。

- **D.** Oui : volatile rend toutes les écritures atomiques.

  足够：volatile 会让所有写操作变为原子操作。

Réponse / 答案：B

`volatile` impose des contraintes au traitement des accès par le compilateur, mais ne remplace pas un protocole de synchronisation ou une opération atomique.

volatile 对编译器处理访存施加约束，但不能替代同步协议或原子操作。

Repères / 复习讲义：2.10。

TD, TP et bases des API / TD、TP 与 API 基础

Q076

Prioritaire / 核心

Que demande l'option `-c` dans `nvcc -c tp1_check.cu` ?

`nvcc -c tp1_check.cu` 中的 `-c` 要求完成哪一步?

- **A.** Produire directement un exécutable entièrement lié.

  直接生成完成链接的可执行文件。

- **B.** Produire seulement le texte PTX, sans objet hôte.

  只生成 PTX 文本，不生成主机目标文件。

- **C.** Compiler puis exécuter le kernel pour vérifier le résultat.

  编译后运行 kernel，检查结果。

- **D.** Produire un fichier objet sans effectuer l'édition de liens finale.

  生成目标文件，不进行最终链接。

Réponse / 答案：D

L'option `-c` arrête la construction au fichier objet ; l'édition de liens vers l'exécutable constitue une étape distincte.

`-c` 将构建停在目标文件阶段；生成可执行文件还需要链接。

Repères / 复习讲义：4.6。

Q077

Prioritaire / 核心

Dans TP1, `main` lance `__global__ void kernel(int *v) { *v = 1; }` . Quelle partie devient le corps d'une entrée GPU dans le PTX ?

TP1 中， `main` 启动 `__global__ void kernel(int *v) { *v = 1; }` 。哪一部分成为 PTX 中的 GPU 入口函数体?

- **A.** Tout `main` , y compris les appels hôtes et l'affichage final.

  整个 `main` ，包括主机 API 调用和最后的打印。

- **B.** L'expression de lancement hôte, mais pas le corps de `kernel` .

  只有主机上的启动表达式，不包括 `kernel` 的函数体。

- **C.** Uniquement les appels `cudaMalloc` et `cudaMemcpy` de `main` .

  只有 `main` 中的 `cudaMalloc` 和 `cudaMemcpy` 调用。

- **D.** Le corps de `kernel` , qui écrit 1 à l'adresse reçue.

  `kernel` 的函数体，即向传入地址写入 1。

Réponse / 答案：D

Le corps du kernel est destiné au GPU. Dans cet exemple, `main` et le code déclenchant le lancement restent côté hôte.

kernel 函数体在 GPU 上执行；本例中的 `main` 和发起 kernel 启动的代码位于主机端。

Repères / 复习讲义：4.6。

Q078

Prioritaire / 核心

Quelle association décrit correctement les fichiers produits par la chaîne CUDA ?

下面哪组对应关系正确描述 CUDA 编译产物?

- **A.** `.ptx` : code machine CPU ; `.cubin` : source CUDA non compilée.

  `.ptx`：CPU 机器码；`.cubin`：未编译的 CUDA 源码。

- **B.** `.ptx` : code machine GPU définitif ; `.cubin` : texte intermédiaire portable.

  `.ptx`：最终 GPU 机器码；`.cubin`：可移植的文本中间表示。

- **C.** `.ptx` : représentation intermédiaire textuelle ; `.cubin` : code machine pour une cible GPU.

  `.ptx`：文本中间表示；`.cubin`：面向某个 GPU 目标的机器码。

- **D.** `.ptx` : journal d'exécution ; `.cubin` : fichier de résultats numériques.

  `.ptx`：运行日志；`.cubin`：数值结果文件。

Réponse / 答案：C

PTX décrit une cible virtuelle sous forme textuelle. Un cubin contient du code machine destiné à une architecture GPU donnée.

PTX 是面向虚拟目标的文本表示；cubin 包含针对特定 GPU 架构的机器码。

Repères / 复习讲义：4.6。

Q079

Prioritaire / 核心

Dans les cibles historiques `compute_30` et `sm_30`, que signifient le nombre et les deux préfixes ?

历史目标名 `compute_30` 和 `sm_30` 中，数字和两个前缀各表示什么?

- **A.** 30 SM physiques ; `compute` désigne la mémoire et `sm` les instructions.

  30 个物理 SM；`compute` 表示内存，`sm` 表示指令。

- **B.** Capacité de calcul 3.0 ; `compute` désigne une cible virtuelle et `sm` une cible machine.

  计算能力 3.0；`compute` 表示虚拟目标，`sm` 表示机器目标。

- **C.** CUDA Toolkit 3.0 ; les deux préfixes désignent le même format de fichier.

  CUDA Toolkit 3.0；两个前缀表示相同文件格式。

- **D.** 30 threads par bloc ; `compute` désigne le CPU et `sm` le GPU.

每块 30 个线程； `compute` 表示 CPU， `sm` 表示 GPU。

Réponse / 答案：B

Ces noms désignent des cibles de compilation. Le suffixe 30 signifie une capacité de calcul 3.0, et ne décrit ni la version du Toolkit ni le nombre de threads.

这些是编译目标名。后缀 30 表示计算能力 3.0，不表示 Toolkit 版本或线程数。

Repères / 复习讲义：4.6。

Q080

Complément / 补充

Une matrice de 8 lignes et 8 colonnes est stockée par lignes dans un tableau contigu. Avec des indices commençant à 0, quel est l'indice de l'élément `(row=2, col=5)` ?

一个 8 行 8 列的矩阵按行连续存储，下标从 0 开始。元素 `(row=2, col=5)` 的一维下标是多少?

- **A.** 16
- **B.** 42
- **C.** 25
- **D.** 21

Réponse / 答案：D

Le stockage par lignes donne `row*N+col` , donc `2*8+5=21` .

行主序下标为 `row*N+col` ，因此是 `2*8+5=21` 。

Repères / 复习讲义：4.7。

Q081

Prioritaire / 核心

Pour calculer `C[row,col]` dans `C = A*B` , les matrices carrées de taille N sont stockées par lignes. Quel produit doit être accumulé à l'itération k ?

计算 `C=A*B` 的 `C[row,col]` ，三个 N 阶方阵均按行存储。累加循环第 k 次应该计算哪个乘积?

- **A.** `A[row*N+k] * B[col*N+k]`
- **B.** `A[row*N+k] * B[k*N+col]`
- **C.** `A[row*N+col] * B[k*N+k]`
- **D.** `A[k*N+row] * B[k*N+col]`

Réponse / 答案：B

Le produit matriciel combine la ligne `row` de A avec la colonne `col` de B : la dimension k parcourt ces deux ensembles.

矩阵乘法取 A 的第 row 行与 B 的第 col 列做内积，k 遍历二者。

Repères / 复习讲义：4.7。

Q082

Prioritaire / 核心

Pour `C ← αAB + βC` avec `β ≠ 0` et une valeur initiale arbitraire de C, pourquoi faut-il rendre cette valeur initiale disponible au kernel ?

计算 `C ← αAB + βC`，其中 `β ≠ 0` 且 C 的初值任意。为什么 kernel 需要获得 C 的初值?

- **A.** C sert uniquement à transmettre les dimensions de A et B.

  C 仅用于向 kernel 传递 A、B 的维度。

- **B.** Le terme `βC` lit l'ancienne valeur de chaque élément de C.

  `βC` 项需要读取 C 各元素的旧值。

- **C.** Le produit `AB` ne peut pas être calculé sans stocker A dans C.

  计算 `AB` 前必须先把 A 存入 C。

- **D.** Le kernel déduit automatiquement α et β à partir de C.

  kernel 会根据 C 自动推导 α 和 β。

Réponse / 答案：B

C est à la fois une entrée et une sortie de cette opération. Le nouveau résultat dépend explicitement de sa valeur précédente.

这项运算中 C 同时是输入和输出，新结果明确依赖旧值。

Repères / 复习讲义：4.7。

Q083

Prioritaire / 核心

On retient uniquement le terme dominant $2N^3$ pour un produit de matrices N×N, avec une multiplication et une addition comptées comme deux FLOP. Pour N=100, quel est ce nombre de FLOP ?

N×N 矩阵乘法只按主项 $2N^3$ 计数，一次乘法和一次加法合计两个 FLOP。N=100 时，该运算量是多少 FLOP?

- **A.** 20 000 FLOP
- **B.** 1 000 000 FLOP
- **C.** 200 000 000 FLOP
- **D.** 2 000 000 FLOP

Réponse / 答案：D

Il y a N^2 sorties et N étapes par sortie ; le compte demandé vaut $2*100^3=2\ 000\ 000$ FLOP. C'est un travail, pas un débit.

共有 N^2 个输出，每个输出计算 N 轮，按题定口径得到 $2*100^3=2\ 000\ 000$ FLOP。这是工作量，不是吞吐率。

Repères / 复习讲义：4.8。

Q084

Prioritaire / 核心

Un kernel effectue 2×10^9 FLOP en 4 ms. Quel débit cela représente-t-il, avec 1 GFLOP=10^9 FLOP ?

一个 kernel 在 4 ms 内完成 2×10^9 FLOP。按 1 GFLOP=10^9 FLOP，吞吐率是多少?

- **A.** 0,5 GFLOP/s

  0.5 GFLOP/s
- **B.** 500 GFLOP/s
- **C.** 8 000 GFLOP/s
- **D.** 8 GFLOP/s

Réponse / 答案：B

Il faut convertir 4 ms en 0,004 s : $(2\times10^9)/0{,}004=500\times10^9$ FLOP/s.

先将 4 ms 换为 0.004 s： $(2\times10^9)/0.004=500\times10^9$ FLOP/s。

Repères / 复习讲义：4.8。

Q085

Prioritaire / 核心

Pour un même résultat, le CPU prend 80 ms. Le traitement GPU prend 4 ms pour le kernel seul et 10 ms pour le périmètre kernel+transferts choisi. Quel est le speedup sur ce dernier périmètre ?

计算同一结果时，CPU 用时 80 ms。GPU 的 kernel 单独用时 4 ms，题目选定的“kernel+传输”范围总用时 10 ms。按后一个范围计算，加速比是多少?

- **A.** 20
- **B.** 2,5

  2.5
- **C.** 0,125

  0.125
- **D.** 8

Réponse / 答案：D

Le speedup demandé est `t_CPU/t_GPU=80/10=8` . La valeur 20 correspondrait au kernel seul et à un autre périmètre.

题目要求的加速比为 `t_CPU/t_GPU=80/10=8` 。20 对应只测 kernel 的另一种口径。

Repères / 复习讲义：4.8。

Q086

Complément / 补充

On mesure uniquement le kernel, après les allocations et les transferts H2D, et avant le retour D2H. Quelle description de cette mesure est correcte ?

完成分配和 H2D 传输后，仅对 kernel 计时，并在 D2H 回传前结束计时。如何描述这项测量才准确?

- **A.** Elle mesure le kernel et exclut les allocations ainsi que les transferts H2D/D2H.

  它测量 kernel，排除了分配以及 H2D/D2H 传输。
- **B.** Elle inclut automatiquement tous les transferts déclenchés avant le kernel.

  它自动包含 kernel 之前已经发起的所有传输。
- **C.** Elle mesure nécessairement le temps total du programme GPU.

  它必然等于整个 GPU 程序的总时间。
- **D.** Elle suffit pour annoncer un speedup incluant allocations et communications.

  仅凭它就可以报告包含分配和通信的加速比。

Réponse / 答案：A

Le périmètre chronométré fixe l'interprétation du résultat. Un temps kernel seul ne donne pas le coût complet de la solution GPU.

计时范围决定结果的含义。单独的 kernel 时间无法给出 GPU 方案的完整成本。

Repères / 复习讲义：4.8。

Q087

Prioritaire / 核心

Pour un élément de `C ← αAB + βC`, on a `(AB)[i,j]=5`, `α=2`, `β=3` et `C_initial[i,j]=4`. Quelle est la nouvelle valeur de C[i,j] ?

计算 `C ← αAB + βC` 的一个元素，已知 `(AB)[i,j]=5`、`α=2`、`β=3`、`C_initial[i,j]=4`。新的 C[i,j] 是多少?

- **A.** 30
- **B.** 22
- **C.** 10
- **D.** 19

Réponse / 答案：B

La mise à jour donne `2*5+3*4=22` ; le terme contenant l'ancienne valeur de C doit être conservé.

更新结果为 `2*5+3*4=22`，不能漏掉含 C 初值的那一项。

Repères / 复习讲义：4.7。

Q088

Complément / 补充

Le TD2 lance 512 blocs de 256 threads. Si chaque thread réalise 100 tirs, combien de tirs sont effectués au total ?

TD2 启动 512 个块，每块 256 个线程。如果每个线程进行 100 次采样，总采样次数是多少?

- **A.** 131 072
- **B.** 25 600
- **C.** 51 200
- **D.** 13 107 200

Réponse / 答案：D

Le nombre total de threads est `512*256=131072` ; chaque thread effectue 100 tirs, donc `13107200` tirs au total.

总线程数为 `512*256=131072` ，每线程采样 100 次，总计 `13107200` 次。

Repères / 复习讲义：4.5。

Q089

Prioritaire / 核心

Dans la version CUDA demandée au TD2, chaque thread effectue le même nombre de tirs. Quelle organisation correspond au sujet ?

TD2 要求的 CUDA 版本中，每个线程的采样次数相同。哪种任务分配符合题意?

- **A.** Chaque thread calcule une estimation locale de π, l'écrit dans sa case, puis le CPU moyenne les résultats rapatriés.

  每个线程计算局部 π 估计值，写入自己的数组元素，CPU 取回结果后求平均。

- **B.** Chaque bloc produit nécessairement une seule estimation, sans résultat propre à chaque thread.

  每个块必须只产生一个估计值，各线程不保存自己的结果。

- **C.** Chaque thread lance un tir, quel que soit `TRIALS_PER_THREAD` , et le CPU prend le maximum.

  无论 `TRIALS_PER_THREAD` 是多少，每个线程只采样一次，CPU 取最大值。

- **D.** Tous les threads écrivent leur estimation dans une seule case, et le CPU lit la dernière écriture.

  所有线程把估计值写入同一个数组元素，CPU 读取最后一次写入。

Réponse / 答案：A

Le sujet demande un résultat par thread. Le même nombre de tirs par thread permet de faire la moyenne simple de leurs estimations.

题目要求每个线程保存一个结果；各线程采样次数相同，因此可以对这些估计值直接求平均。

Repères / 复习讲义：4.5。

Q090

Complément / 补充

Chaque thread possède un état de générateur pseudo-aléatoire local, mais tous ces états sont initialisés de façon identique et utilisés de la même manière. Quel problème cela pose-t-il pour Monte Carlo ?

每个线程各有一个局部伪随机数状态，但所有状态都采用完全相同的初始化，并以相同方式调用。对 Monte Carlo 计算有什么影响?

- **A.** Le GPU interdit que deux états distincts contiennent les mêmes valeurs initiales.

  GPU 禁止两个独立状态包含相同的初始值。

- **B.** Les threads reproduisent les mêmes suites de points, ce qui réduit la diversité des échantillons.

  各线程重复相同的点序列，降低采样的多样性。

- **C.** Le résultat devient exactement π dès que tous les threads ont terminé.

  所有线程结束时，结果就精确等于 π。

- **D.** La présence de plusieurs threads suffit à rendre les suites indépendantes.

  只要使用多个线程，各随机序列自然独立。

Réponse / 答案：B

Un générateur déterministe reproduit la même suite à état initial et appels identiques. Dans l’exemple, l’identifiant de thread distingue les séquences.

确定性生成器在初始状态和调用过程相同时会生成相同序列。示例使用线程编号区分序列。

Repères / 复习讲义：4.5。

Q091

Prioritaire / 核心

Quelle comparaison entre CUDA Runtime API et Driver API est correcte ?

CUDA Runtime API 与 Driver API 的哪项比较正确?

- **A.** La Runtime API fournit une gestion plus implicite ; la Driver API permet un contrôle plus explicite des ressources et contextes.

  Runtime API 管理较隐式；Driver API 可以更显式地控制资源和上下文。

- **B.** La Runtime API et la Driver API utilisent obligatoirement deux GPU physiques distincts.

  Runtime API 和 Driver API 必须分别使用两个不同的物理 GPU。

- **C.** La Driver API garantit toujours un kernel plus rapide pour le même calcul.

  同样的计算使用 Driver API 就能保证 kernel 更快。

- **D.** La Runtime API s’exécute sur le CPU, tandis que tous les appels Driver s’exécutent dans les threads GPU.

Runtime API 在 CPU 执行，而全部 Driver API 调用在 GPU 线程内执行。

Réponse / 答案：A

La différence porte sur le niveau d’abstraction et le contrôle. Les deux API peuvent accéder au même GPU ; le choix de l’API ne garantit pas un gain de calcul.

区别主要在抽象层次和控制方式。两套 API 可以访问同一 GPU，选择 Driver API 本身并不保证计算加速。

Repères / 复习讲义：3.1。

Q092

Prioritaire / 核心

Quel rôle joue un contexte CUDA dans le modèle du cours ?

按课程模型，CUDA 上下文起什么作用?

- **A.** Il représente uniquement les coordonnées d’un thread dans son bloc.

  仅表示某个线程在块内的坐标。

- **B.** Il organise l’état CUDA d’un GPU, notamment des allocations, des streams et des événements.

  组织某个 GPU 的 CUDA 状态，包括内存分配、stream 和 event 等资源。

- **C.** Il désigne le texte PTX produit par une seule fonction.

  表示由一个函数生成的 PTX 文本。

- **D.** Il est une zone de mémoire partagée propre à un seul bloc.

  是仅属于一个线程块的 shared memory 区域。

Réponse / 答案：B

Le contexte regroupe un état d’exécution et des ressources CUDA. Ce n’est ni un indice de thread ni une mémoire locale à un bloc.

上下文组织 CUDA 执行状态和资源，不是线程编号，也不是块内的一种内存。

Repères / 复习讲义：3.2。

Q093

Complément / 补充

Que fait `cudaSetDevice(1)` lorsqu’il réussit dans un thread hôte ?

某个主机线程成功调用 `cudaSetDevice(1)` 后，发生什么？

- **A.** Il fixe à 1 le nombre de threads du prochain kernel.

  把下一个 kernel 的线程数设为 1。

- **B.** Il sélectionne le GPU d'identifiant visible 1 pour les opérations CUDA ultérieures concernées de ce thread.

  为该主机线程后续相关 CUDA 操作选择可见编号为 1 的 GPU。

- **C.** Il change l'identifiant global de la carte physique pour le mettre à 1.

  把该物理显卡在全系统的编号修改为 1。

- **D.** Il affecte automatiquement le GPU 1 à tous les processus du système.

  自动把整个系统的所有进程都绑定到 GPU 1。

Réponse / 答案：B

La sélection concerne le thread hôte appelant. Elle doit correspondre au GPU voulu lors de la création des ressources et du lancement des kernels.

设备选择作用于调用它的主机线程；创建资源和启动 kernel 时，应确保选择了预期 GPU。

Repères / 复习讲义：3.3。

Q094

Complément / 补充

`cudaGetDeviceCount(&count)` réussit et donne `count=2`. Parmi les affirmations suivantes, laquelle est garantie par ce résultat ?

`cudaGetDeviceCount(&count)` 成功返回 `count=2`。下面哪项可以由这一结果确定？

- **A.** La machine contient exactement deux cartes graphiques physiques, sans autre GPU.

  整台机器恰好只有两张物理显卡，不存在其他 GPU。

- **B.** Les seuls identifiants valides pour ce processus sont 1 et 2.

  该进程只能使用编号为 1 和 2 的设备。

- **C.** Le processus voit deux périphériques CUDA, numérotés 0 et 1 dans son espace de visibilité.

  当前进程可见两个 CUDA 设备，在其可见范围中编号为 0 和 1。

- **D.** Les deux GPU partagent automatiquement toutes leurs allocations.

  两个 GPU 自动共享全部内存分配。

Réponse / 答案：C

Le compte décrit les dispositifs visibles par le processus. Il ne suffit pas à déduire tous les GPU physiques installés ni leurs droits d’accès mémoire.

数量表示当前进程可见的设备，不能据此推断整台机器的物理 GPU 总数或内存访问权限。

Repères / 复习讲义：3.3。

Q095

Complément / 补充

Des données ont été allouées sur le GPU 0. Le thread hôte sélectionne ensuite le GPU 1 avec `cudaSetDevice(1)` . Que peut-on affirmer sur ces données, sans autre opération ?

数据已经分配在 GPU 0 上，随后主机线程用 `cudaSetDevice(1)` 选择 GPU 1。在没有其他操作的情况下，关于原有数据可以确定什么?

- **A.** Les deux GPU peuvent immédiatement accéder aux données, quel que soit le matériel.

  无论硬件条件如何，两个 GPU 都立即获得数据访问权限。

- **B.** Les données du GPU 0 sont automatiquement libérées.

  GPU 0 上的数据自动释放。

- **C.** Le changement de GPU courant ne migre pas ces données vers le GPU 1.

  切换当前 GPU 不会把这些数据迁移到 GPU 1。

- **D.** Toutes les données sont automatiquement copiées vers le GPU 1.

  所有数据自动复制到 GPU 1。

Réponse / 答案：C

`cudaSetDevice` sélectionne un dispositif ; ce n’est ni une copie ni une libération ni une autorisation universelle d’accès distant.

`cudaSetDevice` 用于选择设备，不承担复制、释放或统一授予远程访问权限的作用。

Repères / 复习讲义：3.3。

Q096

Complément / 补充

Sur une plateforme prenant en charge UVA, quelle affirmation décrit correctement cette fonctionnalité ?

在支持 UVA 的平台上，哪项描述正确？

- **A.** Les mémoires physiques du CPU et des GPU deviennent une seule mémoire de même latence.

  CPU 和 GPU 的物理内存合并成延迟完全相同的一块内存。

- **B.** Tout pointeur de mémoire GPU peut être lu directement par le CPU.

  任意 GPU 内存指针都可以由 CPU 直接读取。

- **C.** Les adresses utilisent un espace virtuel unifié, sans garantir que tout processeur puisse déréférencer tout pointeur.

  地址采用统一的虚拟地址布局，但不保证所有处理器都能解引用任意指针。

- **D.** Toute copie entre hôte et GPU est supprimée automatiquement.

  主机与 GPU 之间的所有复制都自动消失。

Réponse / 答案：C

UVA unifie l'identification des adresses. L'emplacement des données, les droits d'accès et les dépendances d'exécution restent des questions distinctes.

UVA 统一地址识别；数据位置、访问权限和执行依赖仍需要分别判断。

Repères / 复习讲义：3.5。

Q097

Prioritaire / 核心

Quelle différence essentielle sépare UVA et une allocation de mémoire unifiée via `cudaMallocManaged` ?

UVA 与通过 `cudaMallocManaged` 创建的统一内存分配，核心区别是什么？

- **A.** UVA concerne l'espace d'adresses ; la mémoire unifiée fournit une allocation accessible au CPU/GPU dont le système gère les accès et mouvements nécessaires.

  UVA 关注地址空间；统一内存提供 CPU/GPU 可访问的分配，并由系统管理必要的访问和数据移动。

- **B.** La mémoire unifiée signifie que CPU et GPU peuvent modifier simultanément la même donnée sans coordination.

  统一内存意味着 CPU 和 GPU 可以无需协调地同时修改同一个数据。

- **C.** UVA migre automatiquement toutes les allocations ; la mémoire unifiée sert uniquement à renommer les pointeurs.

UVA 自动迁移所有分配；统一内存只用于修改指针名称。

- **D.** Les deux termes sont synonymes et garantissent l’absence de toute migration.

  两个术语完全同义，并保证不会发生任何迁移。

Réponse / 答案：A

Une adresse unifiée n’apporte pas à elle seule la gestion de données d’une allocation managed. La mémoire unifiée n’élimine pas le besoin de respecter l’ordre des accès.

统一地址本身不提供 managed 分配的数据管理机制；统一内存也不会取消对访问顺序的要求。

Repères / 复习讲义：3.5。

Q098

Complément / 补充

Une région target s’exécute effectivement sur un dispositif dont la mémoire est distincte de celle de l’hôte, sans mapping préexistant et sans `nowait`. Elle calcule `c[i]=a[i]+b[i]`. Les entrées sont sur l’hôte, qui lit c après la fin de la région. Quel choix de clauses convient ?

一个 target 区域实际在内存独立于主机的设备上执行，不存在预先建立的映射，也不使用 `nowait`。区域计算 `c[i]=a[i]+b[i]`，输入在主机上，主机在区域结束后读取 c。哪组映射子句合适?

- **A.** `map(from: a[0:N], b[0:N], c[0:N])`
- **B.** `map(from: a[0:N], b[0:N]) map(to: c[0:N])`
- **C.** `map(to: a[0:N], b[0:N]) map(from: c[0:N])`
- **D.** `map(to: a[0:N], b[0:N], c[0:N])`

Réponse / 答案：C

Les entrées a et b doivent être disponibles sur le dispositif. La valeur initiale de c n’est pas utilisée ; la sortie c doit revenir à l’hôte.

a、b 是需要送到设备端的输入；c 的初值不参与计算，只需要将输出取回主机。

Repères / 复习讲义：3.13。

Q099

Prioritaire / 核心

Une région target s’exécute effectivement sur un dispositif dont la mémoire est distincte de celle de l’hôte, sans mapping préexistant. Elle met à jour `y[i]=alpha*x[i]+y[i]`. L’hôte fournit y initial et doit récupérer y final. Quelle clause décrit les besoins de y ?

一个 target 区域实际在内存独立于主机的设备上执行，不存在预先建立的映射，区域计算 `y[i]=alpha*x[i]+y[i]` 。y 的初值由主机提供，主机还需要取回 y 的最终结果。y 应采用哪种映射子句?

- **A.** `map(from: y[0:N])`
- **B.** `map(tofrom: y[0:N])`
- **C.** `map(alloc: y[0:N])`
- **D.** `map(to: y[0:N])`

Réponse / 答案：B

y est lu avant d’être mis à jour, puis son résultat doit être récupéré. `tofrom` exprime ces deux directions.

计算先读取 y 的初值，再更新 y，并且结果需要取回，因此使用 `tofrom` 。

Repères / 复习讲义：3.13。

Q100

Complément / 补充

Dans une clause OpenMP, que désigne `a[0:100]` si a est un tableau de `double` ?

在 OpenMP 子句中，若 a 为 `double` 数组， `a[0:100]` 表示什么?

- **A.** 100 octets à partir de l’adresse de a.

  从 a 起始地址开始的 100 字节。
- **B.** Le seul élément d’indice 100.

  只有下标为 100 的一个元素。
- **C.** 101 éléments, d’indices 0 à 100 inclus.

  101 个元素，包括下标 0 到 100。
- **D.** 100 éléments, d’indices 0 à 99.

  100 个元素，下标为 0 到 99。

Réponse / 答案：D

La notation est début:longueur. La longueur compte des éléments du tableau, indépendamment de leur taille en octets.

该语法为“起始下标:元素个数”，长度以数组元素计数，与每个元素占多少字节无关。

Repères / 复习讲义：3.13。

Revenir au parcours prioritaire / 返回核心题索引